



Tefillah

The Master Guide

A complete A-to-Z reference for building, running, and mastering the Tefillah prayer platform

July 3, 2026

Contents

How to Use This Guide	4
What this guide is	4
What is inside	4
How to read it	4
A word on secrets	4
The golden rule	4
System Overview, Architecture & Technology Stack	5
1. What Tefillah Is (and Who Uses It)	5
2. The Three Surfaces	5
3. Architecture Map	5
4. End-to-End Route Maps	6
5. Complete Technology Stack	8
The Backend, Exhaustively	12
The Backend, Exhaustively	12
The Mobile App (Expo / React Native)	23
The Mobile App (Expo / React Native)	23
The Web App and Admin Panel	30
The web app and admin panel (tefillah-web/)	30
Credentials, Environment Map & Deployment Runbooks	37
Overview: three components, three config surfaces	37
1. Credentials & environment-variable map	37
2. External accounts (who to log into for what)	39
3. Deployment runbooks	40
4. Google Workspace email specifics (and the Resend coexistence rule)	42
5. The AWS clock-skew gotcha (read this before every deploy)	42
The "Become a Master" Playbook	44
The "Become a Master" Playbook	44
Appendix — Command Cheat-Sheet	52
Backend	52
Web (public site + admin panel)	52
Mobile (Expo / EAS)	52
Git	52

Fast fixes 52

CHAPTER 1

How to Use This Guide

What this guide is

This is the single, complete operating manual for **Tefillah** — every surface, every technology, every credential location, and every "where do I change X" you will need to run and grow the platform on your own. It is written to be kept open while you work.

What is inside

1. **Overview, Architecture & Route Map** — the big picture and how a prayer flows end to end, plus the full technology stack.
2. **Backend Deep Dive** — the FastAPI server: every endpoint, the database schema, the security model, AI, email, and push.
3. **Mobile App Deep Dive** — the Expo app's routing, API client, authentication, and notifications.
4. **Web + Admin Deep Dive** — the public website and the neon admin control room.
5. **Credentials, Environment & Deployment** — the full config map and copy-paste deploy runbooks.
6. **Master Playbook** — the "edit X in file Y" table, self-fix troubleshooting, operations runbook, and store-submission checklists.

An **Appendix** at the end gives you a one-page command cheat-sheet.

How to read it

- Use the **bookmarks / outline panel** (or the Contents page) to jump to any chapter or heading — every heading is a clickable outline entry.
- **File paths** are relative to the repo root `D:\tefillah-fixed\tefillah-fixed\` unless stated otherwise.
- **Commands** are copy-paste ready. Backend/git commands assume Git Bash; a few Windows steps call out PowerShell.
- Jump-to-code references are written like `file : function` so you can search the file directly.

A word on secrets

For safety, **actual secret values are never printed in this document** — only the *names* of variables and *where their values live* (`backend/.env`, the Elastic Beanstalk environment, or a provider console). Non-secret identifiers (region, bucket, CloudFront id, domains, project ids) *are* included so the runbooks are usable. To read a real value, open `backend/.env` or the relevant console. Those files are already git-ignored — never commit them.

The golden rule

The backend at `api.tefillah.in` is the one hub every client depends on. When something breaks, check three things in order: (1) the client's API base URL, (2) the backend route in `server.py`, (3) the external service's credential. Ninety percent of issues live in one of those three.

CHAPTER 2

System Overview, Architecture & Technology Stack

1. What Tefillah Is (and Who Uses It)

Tefillah (also spelled Tefilah — the app's display name is "Tefillah", the Android/iOS package id is `com.tefillah.app`) is a full-stack **prayer-request platform**. A person in need writes a prayer request; the system responds instantly with an AI-generated word of comfort and a fitting scripture verse; an admin routes the request to a human **prayer partner**; the partner actually prays over it and marks it done; the requester is kept in the loop.

There are **three kinds of people** on the platform, and each has its own sign-in path and its own home surface:

- **Users** (requesters) — ordinary people who submit prayer requests. They sign up with email + a 6-digit verification code, or with Google. They can browse their prayer history and a community pulse.
- **Prayer partners** (called "partners" in code; older screens say "agent") — vetted volunteers/organizations who receive assigned requests on a dashboard, open them, spend time in prayer, and mark them "prayed". Partner accounts require **admin approval** before they can log in.
- **Admins** — you (the owner) and any co-admins. Admins log in through a separate panel, approve partners, assign prayers to partners, moderate flagged content, view analytics, broadcast notifications, and manage users/partners.

A single human can hold **both** a user and a partner account on the same email and hot-swap between them (see the account-switch route in §4).

2. The Three Surfaces

Tefillah is a monorepo at `D:\tefillah-fixed\tefillah-fixed\` with three independently-deployed front-of-house surfaces plus one backend:

Surface	Folder	Tech	Who it serves	Ships to
Mobile app	<code>frontend/</code>	Expo / React Native	Users + partners	Android AAB/APK, iOS (EAS Build)
Public web + admin	<code>tefillah-web/</code>	Vite + React 19	Users (public site) + admins (<code>/admin</code>)	S3 + CloudFront → <code>tefillah.in</code>
Backend API	<code>backend/</code>	FastAPI (Python)	All of the above	Elastic Beanstalk → <code>api.tefillah.in</code>

Key facts to keep straight:

- **One backend serves all three clients.** The mobile app, the public website, and the admin panel all call the *same* FastAPI server at `https://api.tefillah.in/api`. There is no separate admin backend.
- The **mobile app** is a single Expo codebase that renders native on Android/iOS and can also export to web (`expo export --platform web`), but the *production* website is the separate `tefillah-web/` Vite app, not the Expo web export.
- The **admin panel** is not a separate app — it lives inside `tefillah-web/src/admin/` and is served under the `/admin` path of the same `tefillah.in` site.

3. Architecture Map

The backend is the hub. Every client talks only to `api.tefillah.in`; the backend fans out to the managed services (MongoDB Atlas, Gemini, Firebase FCM, Resend, S3/CloudFront). No client ever talks to MongoDB or Gemini directly.



How to read this while working: if a client is misbehaving, the fault is almost always in one of three places — the client's `API_URL/baseURL`, the backend route in `server.py`, or the external service's credential (an `<EB env var>`). The backend is the single choke point where you can log, patch, and redeploy for *all* clients at once.

4. End-to-End Route Maps

4a. The prayer-request journey (the core flow)

This is the spine of the whole product. Trace it once and you understand Tefillah.

- Submit.** A signed-in user taps "Submit prayer" in the app (or on the web). The client `POSTs` to `/api/prayer/submit` (handler `submit_prayer` in `backend/server.py`) with `{ content, is_anonymous, language, location_* }` and a JWT Bearer token. (Guests without an account hit `/api/prayer/guest-submit` instead, rate-limited harder.)
- Rate-limit + auth.** `get_current_user` validates the JWT; `check_rate_limit("prayer:<ip>", 5)` caps 5 submissions/window or returns HTTP 429.
- LLM comfort + verse.** The handler builds a compassionate Christian system prompt (localized to English/Hindi/Telugu via `lang_map`) and calls `generate_llm_response(...)`, which dispatches to `generate_with_gemini` (default provider). Gemini returns JSON — `category, comfort_message, bible_verse, bible_reference` — parsed by `parse_llm_json`. If the LLM fails or the JSON won't parse, **safe fallbacks** kick in (category `prayer`, Psalm 34:18) so the user always gets a response.
- Store.** A `prayer_doc` (id = `uuid4`) is inserted into the `prayer_requests` MongoDB collection with status: `"pending"`, `assigned_partner_id: None`. Anonymous submissions null-out `user_name/user_email`. The event is written to the activity log via `log_activity("prayer_submitted", ...)`.

5. **Confirm to user.** A confirmation email (gold-branded HTML, showing the category, comfort message, and scripture) is fired to the user through Resend, and the `ComfortResponse` (comfort message + verse) is returned so the app shows it on the confirmation screen immediately.
6. **Assign (admin).** New prayers land as **pending**. An admin opens the admin panel's prayers view and calls `/api/admin/prayers/{prayer_id}/assign` to route it to a chosen partner (there's also `/admin/partners-for-assignment` to list eligible partners, and `/unassign` to reverse it). Status moves to **assigned**; `assigned_partner_id/assigned_at` are set.
7. **Partner sees it.** The partner's dashboard calls `/api/partner/requests` and shows the request. Opening it fires `/api/partner/requests/{prayer_id}/seen`, which stamps `seen_at` (this starts the reflection timer).
8. **Pray + complete.** After spending time in prayer, the partner calls `/api/partner/requests/{prayer_id}/mark-prayed` (handler `mark_prayer_as_prayed`). The backend **enforces a 60-minute minimum** measured from `seen_at` (falls back to `assigned_at` for legacy rows) — mark it too early and you get an HTTP 400 telling you how many minutes remain. On success: `status` → "prayed", `prayed_at` set, and the partner's stats (`prayers_handled`, `total_prayer_time_minutes`, `response_rate`) are recomputed.
9. **Notify.** Notifications flow through the `notifications` collection and Firebase FCM push (partner-side `/api/partner/notifications`, user-side `/api/user/notifications`). Moderation hooks exist too: partners can `/report` a request or `/block-user`, and any prayer can be `/flagged`.

4b. Auth route map

Step	Endpoint	Notes
Email signup	POST <code>/api/auth/register</code>	Creates user, emails a 6-digit code via Resend
Verify email	POST <code>/api/auth/verify-email</code>	Confirms the 6-digit code; <code>resend-verification</code> re-sends
Login	POST <code>/api/auth/login</code>	Returns a <code>TokenResponse</code> (JWT, HS256, 24h). Per-identity failed-login lockout (10 tries → 30-min lock; admins 5 → 1h)
Who am I	GET <code>/api/auth/me</code>	Resolves the current account from the Bearer token
Forgot password	POST <code>/api/auth/forgot-password</code>	Emails a 6-digit reset code
Reset password	POST <code>/api/auth/reset-password</code>	Consumes the code, sets new password (min 6 chars)
Change password	PUT <code>/api/auth/change-password</code>	Requires current password (authenticated)
Google sign-in	POST <code>/api/auth/social</code>	Client gets a Firebase ID token, backend verifies it and issues its own JWT
Complete social profile	POST <code>/api/auth/social/complete</code>	First-time Google users finish their profile

Google sign-in flow in words: the client (mobile via `@react-native-google-signin`, web via the Firebase web SDK) authenticates against Firebase and receives a **Firestore ID token**; it sends that token to `/api/auth/social`; the backend **verifies the token server-side** (a v33 security fix closed a forged-token bypass) and then mints its own Tefillah JWT. From that point on, every client uses the Tefillah JWT — Firebase is only the front-door identity check.

Partner and admin auth are parallel tracks: `POST /api/partner/register` + `POST /api/partner/login` (login is gated on admin approval), and `POST /api/admin/login` (with `/admin/create-first-admin` to bootstrap the

very first admin using the `ADMIN_SECRET`).

4c. Account-switch route (user ■ partner)

A person with both a user and a partner account on the same email can flip between them without re-logging-in:

- `GET /api/auth/switch-availability` — returns `{ can_switch, target_type }` by looking up the signed-in person's counterpart account (`_counterpart_account`) and checking both are verified (`_switchable`).
- `POST /api/auth/switch-account` — if switchable, it mints a **fresh JWT for the counterpart account** and returns it plus the sanitized `account` object (password hash, verification/reset codes, and `fcm_token` are stripped out). The client swaps in the new token and re-renders as the other role. Both accounts must be verified or you get 403.

5. Complete Technology Stack

Verified against `backend/server.py`, `frontend/package.json` + `frontend/app.json`, `tefillah-web/package.json`, and `deploy-backend.sh`.

Languages

Layer	Technology	Version	Where
Backend	Python	3.11.x	<code>backend/server.py</code>
Mobile	TypeScript	~5.9.2	<code>frontend/</code>
Web/Admin	TypeScript	~5.9.3	<code>tefillah-web/</code>
Config/scripts	JavaScript, Bash	—	<code>deploy-backend.sh</code> , <code>expo scripts</code>

Backend (`backend/server.py`, single-file FastAPI)

Concern	Technology	Notes
Web framework	FastAPI	All routes on an <code>APIRouter</code> mounted under <code>/api</code>
ASGI server	Uvicorn (behind nginx)	Elastic Beanstalk Python platform
Mongo driver	Motor (async) over PyMongo	<code>AsyncIOMotorClient</code> , tz-aware codec
Auth tokens	PyJWT	HS256, 24h expiry; secret in <code>JWT_SECRET</code>
Password hashing	bcrypt	direct <code>bcrypt</code> module
Firebase Admin	<code>firebase-admin</code>	FCM push + Google ID-token verify
HTTP client	<code>httpx</code> (async)	outbound calls to Gemini / Resend
AWS SDK	<code>boto3</code>	lazy S3 client, uses EB instance role
Middleware	CORS, TrustedHost, custom SecurityHeaders	whitelisted origins + HSTS/CSP/X-Frame

Security-relevant behavior baked into startup: the server **refuses to boot** on the built-in default `JWT_SECRET` if it detects a production database (`mongodb+srv`, or any non-localhost Mongo), and likewise for `ADMIN_SECRET` when `PRODUCTION/RAILWAY_ENVIRONMENT` is set. Rate limiting is in-memory (20 req/60s general, 5/60s auth) — not multi-process safe, a known limitation.

Database

Layer	Technology	Where
Primary datastore	MongoDB Atlas (mongodb+srv://...)	URI in MONGO_URL <EB env var>; DB name in DB_NAME (prod = tefillah)

Collections include `users`, `partners`, `admins`, `prayer_requests`, `notifications`, `llm_logs`, `activity_logs`, and `prayer cells`.

AI / LLM

Concern	Technology	Notes
Default provider	Google Gemini (REST)	LLM_PROVIDER=gemini, model gemini-2.5-flash, key in GEMINI_API_KEY
Endpoint	generativelanguage.googleapis.com/v1beta	called via <code>generate_with_gemini</code>
Switchable alternates	OpenRouter (deepseek), Ollama (local deepseek-r1:8b)	selected by LLM_PROVIDER; dispatch in <code>generate_llm_response</code>

The provider is fully switchable by env var; Gemini is the live default. (Older memory notes referencing Ollama/OpenRouter as the primary are stale.)

Mobile app (frontend/ — Expo)

Concern	Technology	Version
Runtime	Expo SDK	~54.0.35
Framework	React Native	0.81.5 (new architecture on)
UI runtime	React	19.1.0
Routing	expo-router (file-based)	~6.0.24, typed routes
State	Zustand	^5.0.10
HTTP	axios	^1.13.2 → https://api.tefillah.in/api
Google sign-in	@react-native-google-signin/google-signin	^16.1.2
Firebase	firebase	^12.10.0
Push	expo-notifications + FCM (google-services.json)	~0.32.17
Storage/secure	expo-secure-store, async-storage	JWT under <code>auth_token</code>
i18n	i18next / react-i18next	EN / HI / TE
Extras	reanimated, gesture-handler, image-picker, location, linear-gradient	—

Public web + admin (tefillah-web/ — Vite)

Concern	Technology	Version
Build tool	Vite	^8.0.1
UI	React	^19.2.4
Styling	Tailwind CSS v4 (@tailwindcss/vite)	^4.2.2

Concern	Technology	Version
Routing	react-router-dom	^7.13.2
State	Zustand	^5.0.12
HTTP	axios	^1.14.0 → VITE_API_URL or https://api.tefillah.in/api
Auth	firebase web SDK	^12.15.0 (Google sign-in)
Icons	lucide-react	^1.7.0
Push	FCM web push (src/lib/webPush.ts)	—

Web JWT is stored in `localStorage` under `tefillah:auth_token`; a 401 response interceptor auto-clears auth and bounces to login.

Infrastructure & external services

Concern	Technology	Non-secret identifiers
API hosting	AWS Elastic Beanstalk	region <code>ap-south-1</code> ; app <code>tefillah-api</code> ; env <code>tefillah-api-prod-v2</code> ; domain <code>api.tefillah.in</code>
Static web / avatars	AWS S3 + CloudFront	bucket <code>tefillah-web-prod</code> ; site <code>tefillah.in</code> ; avatars under <code>/avatars/...</code>
DNS	AWS Route53	<code>tefillah.in</code> zone
Mobile builds	EAS Build	Expo owner <code>tefillah</code> ; projectId <code>22c2d81a-4192-419e-973e-e2112b437e07</code>
Transactional email	Resend	sender <code>admin@tefillah.in</code> ; needs a browser User-Agent to clear Cloudflare
Business email	Google Workspace	<code>admin@tefillah.in</code> ; MX/SPF/DKIM in Route53
Push	Firebase Cloud Messaging	<code>google-services.json</code> (Android) + web VAPID

Where secret values live (never in this guide)

Do **not** transcribe actual values; they live in these places:

- `backend/.env` (local dev) and the **Elastic Beanstalk environment** (production): `MONGO_URL`, `DB_NAME`, `JWT_SECRET`, `ADMIN_SECRET`, `GEMINI_API_KEY`, `RESEND_API_KEY`, `SENDER_EMAIL`, `FIREBASE_ADMIN_CREDENTIALS`, `ALLOWED_ORIGINS`, `S3_AVATAR_BUCKET`, `AWS_S3_REGION`.
- Firebase service-account private key → the JSON file referenced by `FIREBASE_ADMIN_CREDENTIALS` (packaged into the EB bundle as `firebase-credentials.json`).
- Provider consoles hold their own keys: Google AI Studio (Gemini), Resend dashboard, Firebase console, AWS IAM/EB.

Deployment quick-reference

- **Backend:** `bash deploy-backend.sh` from repo root. It restages `backend/server.py` into `_eb_build/`, rebuilds `tefillah-api-v4.zip`, hard-verifies the packaged `server.py` md5 matches source and contains no Twilio references, uploads to the EB S3 bucket, creates an application version, updates env `tefillah-api-prod-v2`, polls health, then smoke-tests `https://api.tefillah.in/api/health`.

- **Mobile:** EAS Build under the tefillah Expo account (app.json → owner tefillah, package com.tefilah.app).
- **Web/Admin:** npm run build in tefillah-web/ (tsc -b && vite build) → sync dist/ to S3 tefillah-web-prod → CloudFront invalidation.

CHAPTER 3

The Backend, Exhaustively

The Backend, Exhaustively

Everything server-side lives in **one file**: `D:/tefilah-fixed/tefilah-fixed/backend/server.py` (~5122 lines). It is a FastAPI app served by uvicorn on port 8001, talking to MongoDB via Motor (async). There is no ORM, no separate models package, no separate routers file — the whole API, its security, its LLM/email/push integrations, and its data models are in this single module. This section maps it end to end so you can jump straight to any function by name or line number.

Companion files in `backend/`:

- `.env` — the real local/dev secrets (masked here as `<set in backend/.env>`).
- `.env.example` — the template of every env var.
- `firebase-credentials.json` — the FCM service-account key (used only if `FIREBASE_ADMIN_CREDENTIALS` points at it).
- `venv/` — the Python virtualenv.

*Production reality (see the Deployment section): the real `MONGO_URL`, `JWT_SECRET`, `DB_NAME=tefilah`, API keys, etc. live in the **Elastic Beanstalk environment**, not in this local `.env`. The local `.env` is dev/localhost only.*

1. How `server.py` is organized

Read top to bottom, the file has these bands. Line numbers are approximate anchors so you can jump:

Band	Lines	What lives here
Imports	1–34	FastAPI, Motor, bcrypt, jwt (PyJWT), httpx, firebase_admin, hmac, csv, boto3 (lazy)
Config / env	36–170	Mongo client + db, JWT config + fail-closed secret guard , LLM provider config, Resend/email config, S3/avatar config, Firebase init, rate-limit + lockout constants
App + middleware setup	172–195	<code>FastAPI(...)</code> (docs disabled unless <code>ENABLE_DOCS=true</code>), <code>api_router = APIRouter(prefix="/api")</code> , <code>HTTPBearer(auto_error=False)</code> , logging
Security helpers	197–288	<code>sanitize_input</code> , <code>check_rate_limit</code> , <code>check_login_lockout</code> , <code>record_failed_login</code> , <code>reset_failed_logins</code> , <code>progressive_login_delay</code> , <code>check_email_action_limit</code> , <code>get_client_ip</code>

Band	Lines	What lives here
Pydantic models	290–620	All request/response schemas (UserCreate, PartnerCreate, PrayerRequestCreate, AdminCreate, TokenResponse, etc.)
Auth/crypto helpers	622–796	hash_password, verify_password, create_token, decode_token, the get_current_* dependencies, log_activity, log_llm_usage, check_admin_permission
Integrations	798–892	send_fcm_push, generate_verification_code, send_email, _account_email_html, send_account_notice
LLM layer	893–1210	generate_with_ollama, generate_with_openrouter, generate_with_gemini, generate_llm_response (dispatcher), parse_llm_json
CORS + security headers + error handler	1148–1240	CORS allowlist, SecurityHeadersMiddleware, global exception handler
Verse pool	1240–1335	_refill_verse_pool, _VERSE_POOL, generate_bible_verse
Route groups	1336–5100	Health/root, then Auth, Partner, Admin, User/Misc, Prayer (see catalog below)
App wiring + lifecycle	5103–5122	app.include_router(api_router), startup_db_client (index creation), shutdown_db_client

To catalog every route yourself: `grep -n '@api_router\.' backend/server.py`. Every endpoint is decorated `@api_router.<method>("/path")`, and the router prefix `/api` means the live path is `/api/<path>`.

2. Complete API endpoint catalog

Paths below are shown **without** the `/api` prefix, but every live URL is `https://api.tefillah.in/api/<path>`. "Auth" column: **None** = public, **User** = `get_current_user`, **Partner** = `get_current_partner`, **Admin** = `get_current_admin`, **Super** = `get_current_super_admin`, **Opt** = optional auth (`get_current_user_optional`).

Auth & account (lines ~1353–2301)

Method	Path	Auth	Purpose
POST	<code>/auth/register</code>	None	Create user, send 6-digit verify code email
POST	<code>/auth/login</code>	None	User login → JWT (24h); 3-layer logout
POST	<code>/auth/verify-email</code>	None	Confirm the 6-digit code, mark <code>is_verified</code>
POST	<code>/auth/resend-verification</code>	Opt	Resend verify code

Method	Path	Auth	Purpose
GET	/auth/switch-availability	User	Can this person switch user↔partner?
POST	/auth/switch-account	User	Issue a token for the counterpart account
POST	/auth/forgot-password	None	Set reset code + expiry, email it
POST	/auth/reset-password	None	Verify reset code (constant-time), set new hash
PUT	/auth/change-password	User	Change password (knows old one)
GET	/auth/me	User	Current profile + derived stats
POST	/auth/social	None	Google/Apple sign-in via Firebase token
POST	/auth/social/complete	None	Finish social signup (missing profile fields)

Partner (lines ~2302–2885)

Method	Path	Auth	Purpose
POST	/partner/register	None	Register a prayer partner (starts pending_approval)
POST	/partner/login	None	Partner login → JWT; 3-layer logout
GET	/partner/stats	Partner	Dashboard bucket counts + totals
GET	/partner/requests	Partner	Assigned prayer requests (filterable)
POST	/partner/requests/{id}/seen	Partner	Mark a request as seen (opens the 60-min clock)
POST	/partner/requests/{id}/report	Partner	Report abusive prayer content (UGC moderation)
POST	/partner/block-user	Partner	Block a user; releases their unassigned requests
POST	/prayer/{id}/flag	Partner	Flag AI-generated comfort content
POST	/partner/requests/{id}/mark-prayed	Partner	Mark prayed; enriches user notification via LLM
GET	/partner/notifications	Partner	Partner notification feed
POST	/partner/notifications/{id}/read	Partner	Mark one read
PUT	/partner/profile	Partner	Edit partner profile
POST	/partner/profile/photo	Partner	Upload partner avatar

Admin (lines ~2886–4260, 4661–4787)

Method	Path	Auth	Purpose
POST	/admin/create-first-admin	None*	Bootstrap first super admin (needs ADMIN_SECRET)
POST	/admin/login	None	Admin login → JWT (4h expiry); strict logout
POST	/admin/create-admin	Super	Create another admin
DELETE	/admin/remove-admin/{id}	Super	Remove an admin
GET	/admin/admins	Super	List admins
GET	/admin/stats	Admin	Dashboard landing stats (open to any admin)
GET	/admin/analytics	Admin (view_analytics)	Time-series: signups, prayers, partner regs, completions
GET	/admin/daily-reports	Admin (view_analytics)	Per-day rollups
GET	/admin/users	Admin (manage_users)	List/search users
PUT	/admin/users/{id}	Admin (manage_users)	Edit status/verify/name/email
DELETE	/admin/users/{id}	Admin (manage_users)	Delete a user
GET	/admin/partners	Admin (manage_partners)	List/search partners
PUT	/admin/partners/{id}	Admin (manage_partners)	Edit a partner
POST	/admin/partners/{id}/approve	Admin (manage_partners)	Approve a pending partner
DELETE	/admin/partners/{id}	Admin (manage_partners)	Delete a partner
GET	/admin/prayers	Admin (manage_prayers)	List/search prayer requests
GET	/admin/partners-for-assignment	Admin (manage_prayers)	Candidate partners for a prayer
POST	/admin/prayers/{id}/assign	Admin (manage_prayers)	Assign a prayer to a partner (fires push)
POST	/admin/prayers/{id}/unassign	Admin (manage_prayers)	Return a prayer to the pool
GET	/admin/llm-logs	Admin (view_analytics)	LLM usage log
GET	/admin/activity-logs	Admin (view_analytics)	Audit trail
POST	/admin/notifications	Admin (manage_notifications)	Create an in-app notification
GET	/admin/export/{data_type}	Super	JSON/CSV export
GET	/admin/export-csv/{data_type}	Admin (export_data)	CSV download stream
POST	/admin/create-admin-with-permissions	Super	Create admin with a permission list
PUT	/admin/admins/{id}/permissions	Super	Edit an admin's permissions
GET	/admin/permissions-list	Admin	The 7-permission vocabulary + labels
POST	/admin/bulk/users	Admin (manage_users)	Bulk user action
POST	/admin/bulk/partners	Admin (manage_partners)	Bulk partner action
POST	/admin/bulk/prayers	Admin (manage_prayers)	Bulk prayer action
POST	/admin/push-notification	Admin (manage_notifications)	Broadcast FCM push

Method	Path	Auth	Purpose
POST	/admin/send-email	Admin (manage_notifications)	Email broadcast
POST	/admin/cells	Admin (manage_partners)	Create a prayer cell
POST	/admin/seed-data	Super	Seed demo data

* /admin/create-first-admin is public but gated by the ADMIN_SECRET in the request body; it only works while zero admins exist.

User / profile / misc (lines ~4278–4760)

Method	Path	Auth	Purpose
POST	/user/register-device	User	Store the FCM fcm_token
POST	/user/unregister-device	User	Clear the FCM token
PUT	/user/profile	User	Edit profile (name/phone/location, email-change flow)
POST	/user/verify-email-change	User	Confirm a new email with a code
POST	/user/cancel-email-change	User	Abort a pending email change
DELETE	/me	User/Partner	Hard-delete own account + data (store requirement)
GET	/community/pulse	User	Aggregate "people praying now" stats
POST	/user/profile/photo	User	Upload user avatar (→ S3)
GET	/avatar/{owner_id}	None	Legacy Mongo-served avatar bytes
GET	/user/notifications	User	User notification feed
POST	/user/notifications/{id}/read	User	Mark one read
POST	/user/notifications/read-all	User	Mark all read
GET	/cells	None	Public list of active prayer cells
GET	/verse/generate	None	Instant Bible verse from the in-memory pool
GET	/health	None	Health probe
GET	/	None	Minimal banner (no version leak)

Prayer (lines ~4788–5106)

Method	Path	Auth	Purpose
POST	/prayer/submit	User	Submit prayer; LLM classifies + writes comfort/verse
POST	/prayer/guest-submit	None	Guest prayer submission (rate-limited)
GET	/prayer/history	User	The signed-in user's past prayers

3. Database schema (MongoDB tefillah)

Motor async driver, `db = client.get_database(DB_NAME)`, `tz_aware=True` so all datetimes come back UTC-aware. `_id` is a **string UUID** everywhere (`str(uuid.uuid4())`), never an ObjectId. Indexes are created at startup in `startup_db_client` (line ~5107): unique index on email for users/partners/admins; `user_id/assigned_partner_id/status` on `prayer_requests`; timestamp on `activity_logs` and `llm_logs`; `created_at` on notifications.

`users` — regular app users.

Field	Meaning
<code>_id</code>	UUID
<code>name, email, phone, address</code>	Profile; email lowercased + unique
<code>location_city, location_country</code>	Location
<code>password_hash</code>	bcrypt hash (may be absent for social-only until reset)
<code>is_verified</code>	Email verified via 6-digit code
<code>is_admin</code>	Always false for users
<code>status</code>	<code>active</code> / <code>suspended</code> / <code>disabled</code> (blocks login in real time)
<code>verification_code, verification_expires</code>	Signup email verification
<code>password_reset_code, password_reset_expires, password_reset_attempts</code>	Forgot-password flow
<code>profile_photo_url</code>	CloudFront S3 avatar URL
<code>fcm_token, fcm_updated_at</code>	Push device token
<code>created_at, last_login</code>	Timestamps

`partners` — prayer warriors/agents.

Field	Meaning
<code>_id, name, email, phone</code>	Identity (email lowercased + unique)
<code>password_hash</code>	bcrypt
<code>location_city, location_country, organization, partner_type</code>	Profile (<code>partner_type</code> e.g. <code>prayer_warrior</code>)
<code>cell_id, cell_name</code>	Matched prayer cell (or null)
<code>status</code>	<code>pending_approval</code> → <code>active</code> / <code>disabled</code>
<code>is_verified, is_active</code>	Approval/active flags (both must pass to log in)
<code>blocked_users</code>	Array of user IDs this partner blocked (capped 5000)
<code>prayers_handled, total_prayer_time_minutes, response_rate</code>	Stats
<code>last_active</code>	Heartbeat (updated ≤ once/30s on any auth request)
<code>profile_photo_url, fcm_token, created_at</code>	Media/push/timestamps

`prayer_requests` — user submissions and their lifecycle.

Field	Meaning
<code>_id, user_id</code>	UUIDs (<code>user_id</code> nulled on account deletion)

Field	Meaning
user_name, user_email	Null when is_anonymous
content	The prayer text
is_anonymous	Hides identity from partner
location_city/country/lat/lon	Location
category	LLM-classified: health/family/finance/spiritual/relationship/work/grief/gratitude/other
comfort_message, bible_verse, bible_reference	LLM-generated at submit
status	pending → assigned → prayed; plus flagged
assigned_partner_id/name, assigned_cell_id/name, assigned_at	Assignment
seen_by_partner, seen_at	Seen-tracking (drives the 60-min clock)
submitted_at, prayed_at	Timestamps

admins — platform administrators.

Field	Meaning
_id, name, email	Identity (unique email)
password_hash	bcrypt
is_super_admin	Bypasses every permission check
is_active	Deactivated admins lose access immediately
permissions	Array from the 7-permission vocabulary (or ["all"])
password_reset_*	Reset flow (isolated from user/partner resets)
created_at, last_login	Timestamps

prayer_cells — geographic/org groups: _id, name, location_city, location_country, description, agent_count, request_count, is_active, created_at. Partners are auto-matched to a cell at registration by city+country regex; agent_count is decremented when a partner deletes their account.

activity_logs — audit trail (written by log_activity): _id, action, actor_type, actor_id, actor_name, target_type, target_id, details, ip_address, timestamp.

llm_logs — LLM usage (written by log_llm_usage): _id, request_type, provider, model, prompt_tokens, completion_tokens, total_tokens, duration_ms, user_id, status (success/error), error_message, timestamp.

notifications — in-app alerts: _id, title, message, type, target_type (all/specific), target_ids, is_read, created_at.

avatars — legacy Mongo-stored avatar bytes: _id = owner UUID, data (binary, occasionally legacy base64 string), content_type. Served by GET /api/avatar/{owner_id}. New uploads go to S3 instead (see §8).

4. The security model

JWT (create_token L631, decode_token L643)

- **HS256**, secret JWT_SECRET. create_token(user_id, email, user_type, is_admin, expiration_hours) puts user_id/email/user_type/is_admin/exp/iat in the payload.

- Expiry: **24h** for users/partners (`JWT_EXPIRATION_HOURS = 24`); admin login overrides to **4h** (`create_token(..., expiration_hours=4)` at L2957).
- `decode_token` **pins the algorithm** to `["HS256"]` (blocks `alg:none` and RS/HS confusion) and **requires** `exp`, `iat`, `user_id` claims — a token missing any is rejected. Expired → 401 "Token has expired"; anything else invalid → 401 "Invalid token".
- Dependencies re-load the account from Mongo and **re-enforce status in real time**: `get_current_user` (L659) blocks suspended/disabled; `get_current_partner` (L692) blocks disabled/pending_approval/is_active=False and writes a heartbeat; `get_current_admin` (L717) blocks `is_active=False`; `get_current_super_admin` (L732) additionally requires `is_super_admin`. So revoking access is immediate — you don't have to wait out the token's lifetime.

v33 fail-closed JWT default-secret guard (L44–66)

The built-in default `'tefillah-secret-key-2024-sacred'` is treated as fatal in production. If `JWT_SECRET` still equals the default AND the deploy "looks production" (`RAILWAY_ENVIRONMENT/PRODUCTION` set, or Mongo is `mongodb+srv` / any non-localhost host), the app **raises** `RuntimeError` **and refuses to start**. Local dev (localhost Mongo) still boots with a warning. Escape hatch (not recommended): `ALLOW_DEFAULT_JWT_SECRET=true`. `ADMIN_SECRET` has an analogous guard (L120–125).

Password hashing

`hash_password` / `verify_password` (L622–629) use **bcrypt** directly (`bcrypt.hashpw` / `bcrypt.checkpw`), with a per-hash salt from `bcrypt.gensalt()`. `verify_password` swallows exceptions and returns `False` (so a malformed stored hash can never crash a login). Registration enforces `password` min length 8 in the Pydantic model.

Rate limiting + the 3-layer login lockout

In-memory dicts (not multi-process safe — fine for a single EB instance):

- `check_rate_limit(identifier, limit)` (L222) — sliding 60s window. General cap `RATE_LIMIT_MAX_REQUESTS = 20`; auth `AUTH_RATE_LIMIT = 5`; admin login uses a hard 3.
- **Layer 1 — IP rate limit**: `check_rate_limit("auth:<ip>", ...)` at the top of every login. 429 if exceeded.
- **Layer 2 — per-identity lockout**: `check_login_lockout(identity_key)` (L236) → returns `(is_locked, seconds_remaining)`. `record_failed_login` (L247) counts failures in a 15-min window (`FAILED_LOGIN_WINDOW = 900`) and **locks** after the cap — users/partners: `FAILED_LOGIN_MAX = 10` for `FAILED_LOGIN_LOCK_DURATION = 1800s` (30 min); admins stricter: `ADMIN_FAILED_LOGIN_MAX = 5` for `3600s` (1 h). `reset_failed_logins` clears the record on success. Identity keys are `partner:<email>` / `admin:<email>` etc.
- **Layer 3 — progressive delay**: `progressive_login_delay` (L266) sleeps 0, 1, 2, 4, 8s (capped) based on recent failures — humans barely notice, bots are throttled.
- All three are wired into `login_user`, `login_partner` (L2396), and `login_admin` (L2916). `check_email_action_limit` (L277) is now a **no-op** (returns `True`) — the per-email 3/hour cap was removed by product decision; per-IP limiting still guards signup/reset.

v33 hardening beyond the JWT guard

- **Reset-password isolation** (`reset_password` L1735): the submitted code is compared with `hmac.compare_digest` (constant-time, no timing probe). Wrong guesses increment `password_reset_attempts` across every account on that email holding a live code, and the code is **burned after** `PASSWORD_RESET_MAX_ATTEMPTS = 5` wrong tries (otherwise a 6-digit code is guessable within its 30-min life). **Credential isolation**: a user/partner reset updates only `db.users` + `db.partners` (keeps normal logins in sync, and gives a Google-only account a usable password) — it **never** touches `db.admins`. An admin-matched code updates only the admin record. This blocks a user-account reset from ever seizing admin access.

- **Partner login logout:** `login_partner` gained the identical 3-layer logout as user/admin login (previously it had only IP rate limiting).
- **Blocked-users cap:** `partner_block_user` (L2645) refuses to grow `blocked_users` past `MAX_BLOCKED_USERS = 5000`, so a partner document can't be inflated indefinitely.
- **Admin read-endpoint permission gating:** `check_admin_permission(admin, perm)` (L789) — super admins and admins with "all" bypass; otherwise the named permission must be present or it raises 403 "Insufficient permissions: requires '<perm>'". Applied to every admin read/write endpoint (grep `check_admin_permission(admin,)`, mapped as:

Permission	Guards
<code>manage_users</code>	<code>/admin/users (GET/PUT), bulk/users</code>
<code>manage_partners</code>	<code>/admin/partners (GET/PUT/approve), partners-for-assignment, bulk/partners, /admin/cells</code>
<code>manage_prayers</code>	<code>/admin/prayers, prayers/{id}/assign, /unassign, bulk/prayers</code>
<code>view_analytics</code>	<code>/admin/analytics, daily-reports, llm-logs, activity-logs</code>
<code>manage_notifications</code>	<code>/admin/notifications, push-notification, send-email</code>
<code>export_data</code>	<code>/admin/export-csv/{data_type}</code>
<code>manage_admins</code>	<code>admin-management endpoints (super-admin gated)</code>

`/admin/stats` is intentionally **left open to any admin** (it's the dashboard landing card). The full vocabulary is returned by `GET /admin/permissions-list` (L4038).

Transport & response hardening

- **Docs disabled:** `docs_url/redoc_url/openapi_url` are `None` unless `ENABLE_DOCS=true` — the admin route map isn't handed to attackers.
- **CORS** (L1148): allowlist from `ALLOWED_ORIGINS` env, else a hardcoded list (`tefillah.in`, `www`, `app`, `admin` subdomains, localhost dev ports, two Vercel preview hosts). `allow_credentials=True`, methods `GET/POST/PUT/DELETE/OPTIONS`, headers `Authorization`, `Content-Type`, `Accept`, `X-Admin-Secret`.
- `SecurityHeadersMiddleware` (L1176) adds `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy` (camera/mic/geo off), `HSTS` (2y, preload), a strict CSP (`connect-src 'self' https://*.tefillah.in`), masks the `Server` header to `tefillah`, strips `X-XSS-Protection`, and **upgrades any** `http://` **redirect Location to** `https://` (EB's nginx terminates TLS over HTTP internally, so FastAPI's slash-redirect would otherwise leak an http URL).
- **Global exception handler** (L1211) returns a generic JSON body on any unhandled error — never a traceback, file path, or Mongo detail.
- **Input sanitization:** `sanitize_input` (L199) strips script/style tags, all HTML tags, `javascript:/data:` URIs, event handlers, and HTML entities; it's wired into model `field_validators` (e.g. `UserCreate.name/address`).

5. AI / verse helper (LLM layer)

- **Provider dispatch:** `generate_llm_response` (L1105) routes on `LLM_PROVIDER` (default `gemini`). Order: Gemini (if `GEMINI_API_KEY`) → OpenRouter (if `OPENROUTER_API_KEY`) → Ollama (localhost fallback). Models: `GEMINI_MODEL` (default `gemini-2.5-flash`), `OPENROUTER_MODEL` (`deepseek/deepseek-chat-v3-0324`), `OLLAMA_MODEL` (`deepseek-r1:8b`).
- **Gemini call** (`generate_with_gemini` L991): POSTs to `https://generativelanguage.googleapis.com/v1beta/models/<model>:generateContent?key=<GEMINI_API_KEY>` with temperature `0.7`, topP `0.9`, `maxOutputTokens 2048`, and four `BLOCK_ONLY_HIGH` safety settings. Retries transient **429/503** with exponential backoff (1s/2s/4s, 3 attempts). Extracts `candidates[0].content.parts[0].text` and

usageMetadata token counts.

- **Prayer generation** (submit_prayer L4788): builds a detailed compassionate-companion **system prompt** demanding a strict JSON object {category, comfort_message, bible_verse, bible_reference}, supports response language en/hi/te (Hindi/Telugu), calls the LLM with the raw prayer text, then parse_llm_json (L1114) extracts the JSON (stripping deepseek <think> tags and code fences). If the LLM fails or JSON won't parse, **safe fallbacks** are used (category prayer, a Psalm 34:18 comfort). The result is stored on the prayer_requests doc and emailed to the user.
- **Verse pool** (generate_bible_verse L1311): returns a verse **instantly** from an in-memory _VERSE_POOL seeded with curated fallbacks, and refills asynchronously via _refill_verse_pool when the pool runs low — so the user never waits on the LLM for a verse.
- **Logging**: every LLM call flows through log_llm_usage (L757) → an llm_logs document with provider, model, token counts, duration_ms, user_id, and status/error_message. It is **best-effort** — a logging failure only warns, it never breaks the response.

6. Email (Resend)

- send_email(to_email, subject, body) (L829) POSTs to https://api.resend.com/emails with Authorization: Bearer <RESEND_API_KEY>, from: "Tefillah <admin@tefillah.in>" (SENDER_EMAIL), and an HTML body.
- **The browser User-Agent is mandatory**: Resend's API sits behind Cloudflare, which returns "error 1010" for non-browser UAs. RESEND_USER_AGENT (L93) is a Chrome UA string sent on every call. If emails silently stop, this is the first thing to check.
- If RESEND_API_KEY is unset, send_email logs a warning and returns False (never raises). Most lifecycle emails are dispatched via background_tasks.add_task(send_email, ...) so they don't block the response.
- **Branded lifecycle emails**: _account_email_html (L861) wraps text in the dark/gold Tefillah theme; send_account_notice (L881) sends account-state notices. Coverage: **signup verification** (register), **verify success, forgot/reset password, prayer received** confirmation (submit_prayer), **account deleted** (DELETE /me), and admin-driven **suspend/reactivate/approve** notices plus the /admin/send-email broadcast.

7. Push (FCM via firebase-admin)

- Init at startup (L127): if FIREBASE_ADMIN_CREDENTIALS points at a service-account JSON that exists, firebase_admin.initialize_app runs and _firebase_initialized = True. Otherwise push is disabled with a warning (the app still boots).
- send_fcm_push(tokens, title, body, data) (L798) builds a firebase_messaging.Notification and sends per-token, counting success/failure and warning on individual failures. If Firebase isn't initialized or no tokens, it returns a no-op result — never raises.
- **Device tokens** are stored per account: POST /user/register-device?token=<FCM token> (L4278) writes fcm_token + fcm_updated_at onto the user or partner doc (chosen by _user_type); POST /user/unregister-device clears it.
- **Triggers**: assigning a prayer (/admin/prayers/{id}/assign) pushes to the partner; /admin/push-notification broadcasts; mark-prayed enriches the user's notification. (Historically the mobile app registered an Expo token instead of an FCM token — that mismatch was fixed; register the raw FCM token.)

8. Avatars (S3 primary, Mongo legacy)

- **New uploads** → S3: _save_avatar (L4518) validates the content type (AVATAR_ALLOWED_TYPES = jpeg/png/webp/gif) and size (AVATAR_MAX_BYTES = 3 MB), then put_object into bucket S3_AVATAR_BUCKET

(tefillah-web-prod) under a **timestamped key** `avatars/<owner_id>-<ts>.<ext>` with a 1-year immutable `Cache-Control`. It returns the public URL `https://tefillah.in/<key>` (`AVATAR_PUBLIC_BASE`, served via CloudFront). The previous object is deleted best-effort to avoid orphans. The S3 client (`_get_s3`, L112) is lazy and uses the **EB instance-role credentials** (no keys in env). Region `AWS_S3_REGION = ap-south-1`.

- `POST /user/profile/photo` (L4563) and `POST /partner/profile/photo` call `_save_avatar` and store the URL as `profile_photo_url` on the account.
- **Legacy Mongo path:** `GET /api/avatar/{owner_id}` (L4571) streams bytes from the `avatars` collection (data binary or legacy base64) — kept for back-compat with older records.
- **Deletion:** `_delete_avatar` (L4502) removes both the Mongo `avatars` doc and the S3 object (parsed from the `profile_photo_url`) — invoked from `DELETE /me` so account deletion is complete.

Where things live (quick jump list)

- Config/env & fail-closed guard: `server.py` L36–170
- Security helpers & lockout: L197–288
- Crypto/token/dependencies: L622–796
- LLM layer: L893–1210
- CORS/headers/error handler: L1148–1240
- Full route list: `grep -n '@api_router\.' backend/server.py`
- Startup index creation: L5107

CHAPTER 4

The Mobile App (Expo / React Native)

The Mobile App (Expo / React Native)

This is the Tefillah phone app: an **Expo SDK 54 / React Native 0.81** project using **expo-router** (file-based routing) and **Zustand** for state. It is a single codebase that serves three audiences — regular praying **users**, **prayer partners** (also called "agents"), and a legacy agent alias. It also builds to web via `react-native-web`, but the public site lives in a separate project (`tefillah-web/`); this app is primarily the Android/iOS app published as `com.tefillah.app`.

Root on disk: `D:/tefillah-fixed/tefillah-fixed/frontend/`

1. Project layout — what each top folder holds

Path	Holds
<code>frontend/app/</code>	The route tree (<code>expo-router</code>). Every file here is a screen or a layout.
<code>frontend/src/api/</code>	<code>client.ts</code> — the single <code>axios</code> API client + all endpoint wrappers.
<code>frontend/src/store/</code>	Zustand stores: <code>authStore.ts</code> , <code>themeStore.ts</code> , <code>languageStore.ts</code> .
<code>frontend/src/lib/</code>	Cross-cutting helpers: <code>firebase.ts</code> (Google sign-in), <code>secureStorage.ts</code> , <code>socialAuth.ts</code> , <code>alerts.ts</code> .
<code>frontend/src/utils/</code>	<code>pushNotifications.ts</code> (FCM registration), <code>notificationPrefs.ts</code> (local on/off toggle).
<code>frontend/src/components/</code>	Reusable UI: <code>Button</code> , <code>Input</code> , <code>Header</code> , <code>Logo</code> , <code>PrayerBillboard</code> , <code>SocialAuthButtons</code> , <code>ThemeToggle</code> , <code>ErrorBoundary</code> , <code>LegalPage</code> .
<code>frontend/src/i18n/</code>	<code>i18next</code> setup (<code>index.ts</code>) + <code>locales/en.json</code> , <code>hi.json</code> , <code>te.json</code> .
<code>frontend/src/constants/</code>	<code>theme.ts</code> — <code>COLORS</code> , <code>FONTS</code> , <code>SPACING</code> , <code>BORDER_RADIUS</code> constants imported by screens.
<code>frontend/src/data/</code>	<code>legalContent.ts</code> — inline Terms / Privacy text shown in modals.
<code>frontend/assets/</code>	App icon, splash, adaptive icon, favicon (referenced from <code>app.json</code>).
<code>frontend/app.json</code>	Expo app config (package, permissions, plugins, <code>versionCode</code>).
<code>frontend/eas.json</code>	EAS Build profiles (preview/production).
<code>frontend/google-services.json</code>	Firebase Android config — makes FCM push + native Google sign-in work.

Note: `src/App.js`, `src/index.js`, `src/App.css`, `src/index.css` are **vestigial CRA leftovers** — the real entry point is `expo-router/entry` (see `package.json` "main"). Ignore them; routing starts at `app/_layout.tsx` and `app/index.tsx`.

2. The routing map

expo-router turns folders in parentheses into **route groups** that share a layout but don't add a URL segment. There are four groups plus a splash. `typedRoutes` is on (`app.json` → `experiments.typedRoutes`), so route strings are type-checked.

Root / entry

Route	File	Purpose
/ (splash)	<code>app/index.tsx</code>	Animated splash (2.5 s). Waits for auth + language init, then redirects: no language → <code>language-select</code> ; token+partner+verified → partner dashboard; token+user+verified → home; unverified → <code>verify</code> ; no token → <code>landing</code> .
(root layout)	<code>app/_layout.tsx</code>	Wraps everything in <code>ErrorBoundary</code> , sets <code>StatusBar</code> , initializes auth/theme/language stores, and registers push + notification listeners when a token exists . Declares the Stack of the four groups.

`app/(auth)/` — pre-login (Stack, `_layout.tsx`)

Route	File	Purpose
<code>/(auth)/language-select</code>	<code>language-select.tsx</code>	First-run language picker (English / Hindi / Telugu).
<code>/(auth)/landing</code>	<code>landing.tsx</code>	Welcome screen; entry to sign-in/sign-up + Google.
<code>/(auth)/signup</code>	<code>signup.tsx</code>	Regular-user email/password registration.
<code>/(auth)/login</code>	<code>login.tsx</code>	Regular-user email/password login.
<code>/(auth)/verify</code>	<code>verify.tsx</code>	6-digit email-verification code entry; "use different account" resets to landing.
<code>/(auth)/forgot-password</code>	<code>forgot-password.tsx</code>	3-step password reset (email → 6-digit code + new password → success).
<code>/(auth)/complete-profile</code>	<code>complete-profile.tsx</code>	Post-Google gate: collects mandatory phone + city + country (and terms consent) before entering the app.
<code>/(auth)/agent-signup</code>	<code>agent-signup.tsx</code>	Prayer-partner registration.
<code>/(auth)/agent-login</code>	<code>agent-login.tsx</code>	Prayer-partner login.

`app/(main)/` — logged-in USER (Tabs, `_layout.tsx`)

The tab bar shows four tabs: **Home / Pray / History / Menu**. Everything in `HIDDEN_SCREEN`s is a pushed detail screen with `href: null` (no tab, tab bar hidden).

Route	File	Purpose
<code>/(main)/home</code>	<code>home.tsx</code>	Home dashboard + community "prayer pulse" billboard.
<code>/(main)/prayer</code>	<code>prayer.tsx</code>	Submit a prayer request (Pray tab).
<code>/(main)/history</code>	<code>history.tsx</code>	Your past prayers + status.
<code>/(main)/menu</code>	<code>menu.tsx</code>	Menu tab: profile, language cycle, theme, account switch, sign out.

Route	File	Purpose
<code>/main/confirmation</code>	<code>confirmation.tsx</code>	Post-submit confirmation screen.
<code>/main/notifications</code>	<code>notifications.tsx</code>	In-app notification center.
<code>/main/profile-settings</code>	<code>profile-settings.tsx</code>	Edit name/phone/location, photo, email-change flow.
<code>/main/change-password</code>	<code>change-password.tsx</code>	Change password (signed-in).
<code>/main/privacy</code>	<code>privacy.tsx</code>	Redirect shim → <code>privacy-policy</code> .
<code>/main/privacy-policy</code>	<code>privacy-policy.tsx</code>	Privacy policy.
<code>/main/terms</code>	<code>terms.tsx</code>	Terms of service.
<code>/main/community-guidelines</code>	<code>community-guidelines.tsx</code>	UGC community guidelines.

`app/partner/` — **logged-in PARTNER** (`Stack, _layout.tsx`)

The layout **guards access**: if `isInitialized && (!token || userType !== 'partner')` it redirects to `/auth/agent-login`.

Route	File	Purpose
<code>/partner/dashboard</code>	<code>dashboard.tsx</code>	Partner queue: New / Assigned / Pending / Total buckets, mark-seen, mark-prayed, report/block, settings drawer.

`app/agent/` — **legacy alias** (`Stack, _layout.tsx`)

Same guard and a single `dashboard.tsx` (`AgentDashboard`). Kept for backward compatibility; the live partner flow uses `partner`. Note the root `_layout.tsx` Stack only declares `index/auth/main/partner` — `agent` still resolves as a route but is not the primary path.

3. The API client — `src/api/client.ts`

Base URL resolves as: `EXPO_PUBLIC_BACKEND_URL` (baked at build time) with `/api` appended; fallback `https://api.tefillah.in`. So live builds hit `https://api.tefillah.in/api`.

```
const BACKEND_URL = process.env.EXPO_PUBLIC_BACKEND_URL || 'https://api.tefillah.in';
const API_URL = `${BACKEND_URL.replace(/\/$/, '')}/api`;
```

- Instance: `axios.create({ baseURL: API_URL, timeout: 60000 })`.
- **Request interceptor** attaches `Authorization: Bearer <token>` by reading `auth_token` from `secureStorage`.
- **Response interceptor**: on **401** it clears `auth_token` + `user_type` so the next launch re-initializes to logged-out.

Method inventory (grouped exports):

`publicAPI` — `generateVerse(language)`.

`authAPI` — `login`, `register`, `getMe`, `verifyEmail`, `resendVerification`, `switchAvailability`, `switchAccount`, `socialAuth({firebase_token})`, `completeSocialAuth`, `forgotPassword`, `resetPassword`, `changePassword`, `updateProfile`, `verifyEmailChange`, `deleteAccount (DELETE /me)`, `uploadPhoto (multipart → /user/profile/photo)`.

`prayerAPI` — `submit`, `guestSubmit`, `getHistory`, `flagAiContent(prayerId, reason)`.

`communityAPI` — `getPulse()` → `/community/pulse`.

`partnerAPI` — `login`, `register`, `getStats`, `getRequests(status, page, limit)`, `markPrayed(prayerId, minutes)`, `markSeen(prayerId)`, `report(prayerId, reason)`, `blockUser(userId)`,

updateProfile.

agentAPI — thin alias over partnerAPI (getRequests/getStats/markPrayed), for older screens.

notificationsAPI — getAll(page), markRead(id), markAllRead().

deviceAPI — registerToken(token) (token sent as a **query param** to /user/register-device), unregisterToken() → /user/unregister-device.

4. State, theming, i18n

State — Zustand (src/store/). No Redux/Context.

- authStore.ts holds user | partner, token, userType, canSwitch/switchTarget, plus initialize, login, loginAsPartner (+loginAsAgent alias), register, registerAsPartner, verifyEmail, logout, refreshUser, checkSwitch, switchAccount. initialize() reads the stored token, calls getMe, and on failure wipes storage. Token + user_type persist in **secureStorage** (encrypted on native, AsyncStorage on web).
- themeStore.ts exposes mode (dark/light), a full colors object, toggleTheme, setTheme. Persisted to AsyncStorage key tefilah_theme_mode. Use the useTheme() hook.
- languageStore.ts holds language + hasSelectedLanguage; persisted to tefilah_language / tefilah_language_selected.

Theming. Two hand-authored palettes in themeStore.ts: **dark** = pure black + sacred gold #d4af37; **light** = white + teal #0d9488. Screens read const { colors } = useTheme() and style inline. Static constants (COLORS, FONTS, SPACING, BORDER_RADIUS) live in src/constants/theme.ts.

i18n. src/i18n/index.ts wires **i18next + react-i18next** with three bundled locale JSONs. Supported: en, hi (■■■■■■■), te (■■■■■■■); default + fallback en. Screens call const { t } = useTranslation(); t('key'). Changing language goes through languageStore.setLanguage(), which calls i18n.changeLanguage() and persists. The Menu tab cycles languages; first-run uses language-select.

5. Auth on mobile

Native Google sign-in lives in src/lib/firebase.ts:

- On **native** it uses @react-native-google-signin/google-signin (GoogleSignin.configure({ webClientId })) — an in-app account picker, **not** a browser flow. On **web** it uses the Firebase JS SDK popup.
- webClientId = EXPO_PUBLIC_G00GLE_WEB_CLIENT_ID, with a hardcoded public fallback 240620097116-cmjhfmtkkg14gpjv3919imj9l24hm7a1.apps.googleusercontent.com (Web Client IDs are public, so this is safe and guarantees sign-in even if .env fails to load at build).
- The iOS reversed client ID is set in app.json under the google-signin plugin (iosUrlScheme: com.googleusercontent.apps.240620097116-56c70o7cugd0cp71sco1gm6a3tkun).
- signInWithGoogle() returns a Google **ID token**; it handles both v13 and v16 SDK return shapes and treats cancellation as null.
- src/lib/socialAuth.ts::handleSocialAuthFlow(idToken, router) posts it to authAPI.socialAuth (/auth/social), stores the returned JWT + user_type, updates authStore, then **routes**: incomplete profile (missing phone/city/country) → complete-profile; unverified → verify; else home or partner dashboard.
- The Firebase **web config** env vars (EXPO_PUBLIC_FIREBASE_*) are only used on web; native ignores them.

Email verify. After email/password register, verify.tsx collects a 6-digit code → authAPI.verifyEmail({email, code}); resend via resendVerification. On success it routes to home (user) or partner dashboard.

Password reset (email flow). forgot-password.tsx is a 3-step flow: enter email → authAPI.forgotPassword sends a 6-digit code; enter code + new password → authAPI.resetPassword({email, code, new_password}) → success. Signed-in users change password separately via /(main)/change-password →

`authAPI.changePassword.`

Account switch. If the same email exists as both user and partner (verified), `checkSwitch` enables a Menu action that calls `switchAccount()` to swap the JWT and reroute.

6. Push notifications (FCM)

Handled in `src/Utils/pushNotifications.ts`; wired in `app/_layout.tsx`.

- Registration runs **only when a token exists** and the local pref (`getNotificationsEnabled`, default ON, key `tefillah:notifications_enabled`) is not off.
- `registerForPushNotifications()`:
 - Skips simulators (`Device.isDevice`).
 - Requests notification permission.
 - On Android creates channels `default` (MAX) and `prayers` (HIGH).
 - Calls `Notifications.getDevicePushTokenAsync()` — this returns the **native FCM token** on Android (because `google-services.json` is bundled). **This is deliberate:** the backend delivers via the Firebase Admin SDK (`firebase_messaging.send`), which needs a raw FCM token, **not** an Expo push token. Using `getExpoPushTokenAsync()` here would silently break push (that was a real bug — see comments in the file).
 - Registers the token with the backend via `deviceAPI.registerToken(token)` (sent as query param to `/user/register-device`).
- iOS returns an APNs token here which the FCM Admin SDK can't target directly; iOS push is handled separately, so on iOS non-fcm tokens are returned but not registered.
- Deep links from push:** the root layout maps `data.screen` payload keys → routes (`home`, `history`, `prayers`→`history`, `prayer/submit-prayer`→`prayer`, `notifications`, `profile/menu`→`menu`). Keep that map in sync with `app/(main)/`.
- Turning notifications off should call `deviceAPI.unregisterToken()` to clear the token server-side.

7. `app.json` essentials

Field	Value
<code>name / slug</code>	<code>Tefillah / tefillah</code>
<code>version</code>	<code>1.0.0</code> (marketing string)
<code>scheme</code>	<code>tefillah</code> (deep-link scheme)
<code>owner</code>	<code>tefillah</code> (Expo account — builds MUST run under this account)
<code>extra.eas.projectId</code>	<code>22c2d81a-4192-419e-973e-e2112b437e07</code>
<code>newArchEnabled</code>	<code>true</code> (React Native New Architecture / Fabric)
<code>android.package / ios.bundleIdentifier</code>	<code>com.tefillah.app</code>
<code>android.versionCode</code>	<code>4</code>
<code>android.googleServicesFile</code>	<code>./google-services.json</code>
<code>android.allowBackup</code>	<code>false</code>
<code>userInterfaceStyle</code>	<code>automatic</code>

Android permissions granted: `INTERNET`, `ACCESS_NETWORK_STATE`, `CAMERA`, `READ_MEDIA_IMAGES`, `POST_NOTIFICATIONS`, `VIBRATE`, `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`.

Android `blockedPermissions` (explicitly stripped so libraries can't add them — important for Play review): `SYSTEM_ALERT_WINDOW`, `READ_EXTERNAL_STORAGE`, `RECEIVE_BOOT_COMPLETED`, `SCHEDULE_EXACT_ALARM`.

Plugins: expo-router, expo-splash-screen (image + #0f0f1a bg), expo-font, expo-secure-store, expo-notifications (icon + gold #d4af37), @react-native-google-signin/google-signin (iosUrlScheme), expo-image-picker (photos permission text).

iOS infoPlist usage strings: camera, photo library, location-when-in-use, and ITSAppUsesNonExemptEncryption: false.

8. eas.json build profiles

cli.appVersionSource is remote — the build/version number is managed on **EAS servers**, not the local file. Combined with production.autoIncrement: true, production builds auto-bump the version.

Profile	Distribution	Android output	Notes
development	internal	(dev client)	developmentClient: true for debugging.
preview	internal	APK (buildType: "apk")	Installable test build; sideload / internal share.
production	(store)	App Bundle (buildType: "app-bundle", .aab)	For Google Play; autoIncrement: true.

Typical commands (run under the tefillah Expo account): npx eas-cli login then eas build --profile preview --platform android (APK) or eas build --profile production --platform android (.aab). Per project memory, a production .aab was intentionally **held** pending review.

9. Key env vars (EXPO_PUBLIC_*)

All are baked into the JS bundle at build time (the EXPO_PUBLIC_ prefix makes them public — never put secrets here). Set them in frontend/.env (and mirror in EAS if needed).

Var	Purpose	Where value lives
EXPO_PUBLIC_BACKEND_URL	API base; /api appended. Fallback https://api.tefillah.in.	frontend/.env (build env)
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID	Google OAuth Web Client ID for native sign-in. Has a public hardcoded fallback in firebase.ts.	frontend/.env
EXPO_PUBLIC_FIREBASE_API_KEY	Firebase web SDK (web popup only).	frontend/.env
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN	Firebase web auth domain.	frontend/.env
EXPO_PUBLIC_FIREBASE_PROJECT_ID	Firebase project id.	frontend/.env
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET	Firebase storage bucket.	frontend/.env
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	Firebase sender id.	frontend/.env
EXPO_PUBLIC_FIREBASE_APP_ID	Firebase app id.	frontend/.env

Non-env identifiers baked in Google config: the iOS reversed client ID (in app.json) and Android FCM config (in google-services.json). Both are non-secret client identifiers.

Quick "where do I fix X" map

- **Change the API host** → EXPO_PUBLIC_BACKEND_URL (build env) or the fallback in src/api/client.ts.
- **Add/rename an endpoint** → src/api/client.ts (the relevant *API object).

- **Add a screen** → drop a `.tsx` file in the right `app/(group)/` folder; add to that group's `_layout.tsx` if it needs registering; add to `HIDDEN_SCREEN`s in `app/(main)/_layout.tsx` if it should not be a tab.
- **Colors** → `src/store/themeStore.ts`. **Fixed spacing/fonts** → `src/constants/theme.ts`.
- **Translations** → `src/i18n/locales/{en,hi,te}.json`.
- **Push not arriving** → verify FCM token via `getDevicePushTokenAsync` (not Expo token), `google-services.json` present, and backend `firebase_messaging.send`.
- **Google sign-in broken** → `webClientId` in `src/lib/firebase.ts` + Firebase Console Google provider + `app.json` `iosUrlScheme`.
- **Permissions rejected by Play** → `app.json` `android.permissions` / `blockedPermissions`.
- **Version bump for a store release** → managed remotely (`appVersionSource: remote, autoIncrement`); bump `android.versionCode` in `app.json` only if you manage it manually.

CHAPTER 5

The Web App and Admin Panel

The web app and admin panel (tefillah-web/)

tefillah-web/ is the **public website + operator admin console** served at `tefillah.in`. It is a separate codebase from the mobile app (`frontend/`) but talks to the **same FastAPI backend** at `https://api.tefillah.in/api`. Stack: **Vite 8 + React 19 + React Router 7 + Zustand 5 + Tailwind CSS v4 + Firebase Web SDK v12 + axios**. Entry is `src/main.tsx` → `<App/>`; the app mounts into `#root`, wrapped in `<StrictMode>`, and imports `./styles/globals.css` once at the top.

1. Structure and the Tailwind v4 design system

Top-level `src/` layout (`D:/tefillah-fixed/tefillah-fixed/tefillah-web/src/`):

Folder / file	What lives there
<code>App.tsx</code>	The whole router (public + protected + lazy admin)
<code>main.tsx</code>	React mount point
<code>config.ts</code>	Web feature flags (<code>GOOGLE_SIGNIN_ENABLED = true</code>)
<code>pages/</code>	All public + signed-in user + partner page components
<code>admin/</code>	The entire self-contained admin console (lazy-loaded)
<code>components/</code>	Shared UI: Layout, Header, Footer, Logo, ThemeToggle, GoogleSignInButton, NotificationsToggle, PasswordInput, PhoneCodeInput, CountrySelect, ErrorBoundary, ScrollToTop, AboutTefillah, AiContentNotice
<code>api/client.ts</code>	Public axios client + all user/partner/push API groups
<code>auth/ProtectedRoute.tsx</code>	Gate for signed-in user/partner routes
<code>store/authStore.ts, store/themeStore.ts</code>	Zustand stores (session + light/dark theme)
<code>lib/</code>	<code>firebase.ts, socialAuth.ts, webPush.ts, storage.ts</code>
<code>data/countries.ts</code>	Country list for the location selector
<code>styles/globals.css</code>	The public design system (Tailwind v4 + tokens)

Design system — `src/styles/globals.css`. It uses **Tailwind v4** via `@import "tailwindcss";` and `@tailwindcss/vite` (no `tailwind.config.js`; theme is CSS-first). Fonts are registered in an `@theme` block:

- `--font-sans: 'Inter', ...` (body text)
- `--font-serif: 'Cormorant Garamond', ...` (brand/serif headings; the `.font-serif` utility and Tailwind's `font-serif` both map to it)

*Note the header comment: this file was **reconstructed after a crash** (see `project_crash_corruption_2026-07`), so token values are a best-effort brand match — every custom property the app references is defined, but colors may need fine-tuning.*

Theming is by `data-theme` **on** `<html>`. `themeStore.initialize()` runs from `App.tsx` on mount and sets `document.documentElement.dataset.theme`. Two token blocks:

- **Light** (default, `:root / [data-theme='light']`): warm paper `--color-bg:#f6f4ef`, ink text `#1c1b29`, muted gold accent `--color-accent:#b8892b`.
- **Dark** (`[data-theme='dark']`): deep indigo `--color-bg:#0f0f1a`, surface `#1a1a2e`, brighter **gold accent** `#d4af37`.

Key token families every page uses: `--color-bg / -bg-elev / -surface`, `--color-text / -secondary / -muted`, `--color-border / -border-strong`, `--color-accent / -accent-glow`, `--color-success / -warning / -error`, `--radius-md`, `--shadow-md`, `--shadow-glow`, and logo tokens `--logo-disc-outer/-inner/-flame/-flame-droplet`.

Reusable component classes are defined under `@layer` components: `.surface-card`, `.btn-primary` (gold fill), `.btn-ghost`, `.input` (gold focus ring via `--color-accent-glow`), `.eyebrow` (uppercase gold label). Utilities: `.divider-rule` (the "or" line), entrance animations `.anim-fade-up / .anim-slide-in` with `.delay-100/200/300`, and the **breathing logo** trio `.anim-glow-breathe / .anim-icon-breathe / .anim-ring-breathe` (4s loops). Everything is wrapped in `@media` (`prefers-reduced-motion: reduce`) that disables animation.

Layout wrapper — `components/Layout.tsx`. Takes variant: `'public' | 'app'` and `hideNav`. It renders `<Header/> + <main> + optionally <Footer/>`. Rule: the marketing **footer is hidden when** `variant==='app'` (signed-in pages) **and on legal pages** (`hideNav`). `App.tsx` mounts different `<Layout>` wrappers per route group.

2. Public site routes and pages

Routing lives entirely in `src/App.tsx` using `<BrowserRouter>` + nested `<Route element={...}>` groups, with `<ScrollToTop/>` and a top-level `<ErrorBoundary/>` (each page is also individually wrapped in `<ErrorBoundary/>`). Files are in `src/pages/`.

Public / auth / legal (unauthenticated):

Route	File	Purpose
<code>/ (index)</code>	<code>LandingPage.tsx</code>	Marketing landing / hero
<code>/login</code>	<code>LoginPage.tsx</code>	User email+password login (+ Google button)
<code>/signup</code>	<code>SignupPage.tsx</code>	User account creation
<code>/verify</code>	<code>VerifyPage.tsx</code>	Email/account verification step
<code>/complete-profile</code>	<code>CompleteProfilePage.tsx</code>	Fill phone/city/country after social sign-in
<code>/forgot-password</code>	<code>ForgotPasswordPage.tsx</code>	Password reset request
<code>/about</code>	<code>AboutPage.tsx</code>	About Tefillah
<code>/partner/login</code>	<code>PartnerLoginPage.tsx</code>	Prayer-partner login
<code>/partner/signup</code>	<code>PartnerSignupPage.tsx</code>	Prayer-partner (agent) registration
<code>/privacy</code>	<code>PrivacyPage.tsx</code>	Privacy policy (clean layout, <code>hideNav</code>)
<code>/terms</code>	<code>TermsPage.tsx</code>	Terms of service (<code>hideNav</code>)
<code>/delete-account</code>	<code>DeleteAccountPage.tsx</code>	Account-deletion request (store-compliance page, <code>hideNav</code>)

Signed-in USER routes — wrapped in `<ProtectedRoute><Layout variant="app"/></ProtectedRoute>`:

Route	File	Purpose
<code>/home</code>	<code>HomePage.tsx</code>	User home (verse, submit prayer entry)
<code>/prayer</code>	<code>PrayerPage.tsx</code>	Compose/submit a prayer request

Route	File	Purpose
/prayer/confirmation	ConfirmationPage.tsx	Post-submit confirmation
/history	HistoryPage.tsx	Past prayer requests
/menu	MenuPage.tsx	User menu / navigation hub
/profile-settings	ProfileSettingsPage.tsx	Edit name/phone/location/avatar
/app-settings	AppSettingsPage.tsx	Preferences incl. the web-push <NotificationsToggle/>

Signed-in PARTNER route — `<ProtectedRoute requirePartner>`:

Route	File	Purpose
/partner/dashboard	PartnerDashboardPage.tsx	Partner control center (4 buckets + settings drawer)

Fallbacks: `*` → `NotFoundPage.tsx`; `/index` → redirect to `/`. (`ChangePasswordPage.tsx`, `NotificationsPage.tsx`, `LegalLayout.tsx` also exist in `pages/` as supporting components.)

`ProtectedRoute` (`src/auth/ProtectedRoute.tsx`) reads `authStore`; unauthenticated visitors bounce to `/login`, and `requirePartner` enforces partner user-type. The **public axios client** (`src/api/client.ts`) auto-clears the session and redirects to `/login` on any 401, unless the current path is a public/admin path.

Partner dashboard specifics (`PartnerDashboardPage.tsx`, per `project_partner_dashboard`): four stat boxes in fixed order — **New** (`prayers_new`, assigned but not opened), **Assigned** (opened <24h), **Pending** (opened ≥24h, no response), **Total** (all-time). Opening a New request calls `POST /partner/requests/{id}/seen` (`partnerAPI.markSeen`), which starts both the 24h window and the **60-minute minimum-prayer timer** (the "mark prayed" gate). `GET /partner/requests?bucket=new|assigned|pending|prayed` filters; `GET /partner/stats` returns counts. A top-right **Settings gear** opens a right-side `SettingsDrawer` (`.anim-slide-in`) with Edit organisation/name/phone (`PUT /partner/profile`), the notifications toggle, and Sign Out.

3. The lazy-loaded admin panel (`src/admin/`)

The admin console is **code-split out of the public bundle**. `App.tsx` does `const AdminApp = lazy(() => import('./admin/AdminApp'))` and mounts it at `path="admin/*"` inside `<Suspense>` (fallback: a gold spinner on `#0a0a0c`). None of the admin code, its `admin.css`, or its chart code ships to normal visitors.

Routing — `src/admin/AdminApp.tsx`. Mounted at `/admin/*`, so it uses **relative** child paths. On mount it hydrates the admin auth store. Routes:

Path	Page component	Gate
/admin/login	AdminLoginPage.tsx	public
/admin (index)	pages/DashboardPage.tsx	protected
/admin/users	pages/UsersPage.tsx	manage_users
/admin/partners	pages/PartnersPage.tsx	manage_partners
/admin/prayers	pages/PrayersPage.tsx	manage_prayers
/admin/analytics	pages/AnalyticsPage.tsx	view_analytics
/admin/admins	pages/AdminsPage.tsx	super-admin only
/admin/audit-logs	pages/AuditLogsPage.tsx	view_analytics

Anything unknown under `/admin` redirects to `/admin`.

Auth — separate token, separate store. The admin session is deliberately isolated from the user/partner session so they never collide:

- `src/admin/adminApi.ts` creates its own axios instance `adminClient` (baseURL `VITE_API_URL` or `https://api.tefillah.in/api`, 60s timeout). Its `adminStorage` uses distinct `localStorage` keys: `tefillah:admin_token` and `tefillah:admin_user` (the public client uses `tefillah:auth_token`). A request interceptor attaches `Authorization: Bearer <admin token>`. A response interceptor: on 401 it clears the admin token and hard-redirects to `/admin/login` (unless already there).
- `src/admin/adminAuth.tsx` is a Zustand store (`useAdminAuth`) with `admin`, `token`, `initialize()` (hydrate from storage), `login(email, password) → adminApi.login → POST /admin/login`, `logout()` (clears storage), `clearError()`. The `AdminUser` shape includes `is_super_admin`, `is_admin`, and a `permissions: string[]`.
- `src/admin/AdminProtectedRoute.tsx` renders nothing until the store is initialized, then redirects to `/admin/login` if there's no token.
- `AdminLayout.tsx` filters the sidebar nav by permission: super admins (and the wildcard 'all' permission) see everything; otherwise `superOnly` items and each item's `requires` permission are checked so no link dead-ends in a 403. Individual pages re-check (e.g. delete on Prayers is `admin?.is_super_admin` only).

Design system — `src/admin/admin.css` ("control-room" neon theme). Imported by `AdminApp.tsx`, so it only ships in the admin chunk. Everything is scoped under `.admin-root` so it never bleeds into the public site, and it is **always dark**. The canvas is near-black `--bg:#07070c` with layered radial ambient glows (gold, violet, cyan) plus a faint 40px dev-grid, `background-attachment: fixed`. Accent palette: `--gold:#e5b93d` (hero), plus data-viz `--cyan #35e0e0`, `--violet #a78bfa`, `--emerald #38d39b`, `--rose #fb7185`, `--amber`, `--blue`. Glow tokens `--glow-gold / --glow-cyan`. Reusable classes: `.admin-card / .admin-card-hover`, `.admin-hero` (animated gradient hairline border via `admin-border-flow`), `.admin-input(-plain)`, `.admin-btn / .admin-btn-primary` (gold gradient), `.admin-chip`, `.admin-dot` (pulsing status dot), `.admin-table`, `.admin-seg` (segmented period toggle), `.admin-grad-gold` (gold gradient text), `.admin-num` (tabular figures), skeleton `.admin-shimmer`, entrance `.admin-fade-up` with `.admin-d1..d6` delays. All animation respects `prefers-reduced-motion`.

The breathing flame — `src/admin/AdminLogo.tsx`. Renders the exact brand flame (Ionicons v7 flame, the same two SVG subpaths the public `<Logo>` and mobile app use) inside a four-layer badge: (0) soft outer aura, (1) gold conic ring, (2) dark disc, (3) the flame. Layers 0/1/3 gently pulse via `.admin-breathe-glow / -ring / -icon` (4.2s loops defined in `admin.css`, transform/opacity only, reduced-motion safe). Used in the sidebar, mobile top bar, and login screen.

Shared admin primitives — `src/admin/adminUi.tsx` export: `AdminPage`, `AdminPageHeader`, `CountUp` (animated number), `StatusBadge`, `PhoneLink`, `EmailLink`, `Pagination`, `LoadingState`, `ErrorState`, `EmptyState`, and date helpers `fmtDate / fmtDateTime / relTime`.

Each admin page and the endpoints it calls (all via `adminApi` in `adminApi.ts`, hitting `/api/admin/*`):

Page	Shows	Endpoints called
Dashboard (<code>DashboardPage</code>)	Greeting + KPI count-up tiles (users, partners, prayers, pending/assigned/completed, LLM requests/tokens, active-today, new-this-week, suspended counts) + trend charts; a period toggle (day/week/month); manual refresh	GET <code>/admin/stats</code> , GET <code>/admin/analytics?period=</code> (analytics is optional — swallowed on 403 so the dashboard still renders)
Users (<code>UsersPage</code>)	Paginated (20/pg) user table: name/email / phone/location/verified/status/created/last-login; search; suspend↔activate, edit name+email, delete (perm-gated)	GET <code>/admin/users</code> , PUT <code>/admin/users/{id}</code> , DELETE <code>/admin/users/{id}</code>

Page	Shows	Endpoints called
Partners (PartnersPage)	Paginated partner table + online dot (last-active <5min); capacity, prayers handled, response rate; approve, activate/deactivate, edit, delete	GET /admin/partners, PUT /admin/partners/{id}, POST /admin/partners/{id}/approve, DELETE /admin/partners/{id}
Prayers (PrayersPage)	Paginated prayer table with status filter (pending/assigned/prayed/completed) + search; expandable rows; assign to a partner, unassign , delete (super-admin only)	GET /admin/prayers, GET /admin/partners-for-assignment, POST /admin/prayers/{id}/assign?partner_id=, POST /admin/prayers/{id}/unassign, POST /admin/bulk/prayers (delete via {action: 'delete', prayer_ids: [id]})
Analytics (AnalyticsPage)	Time-series (registrations, submissions, completions, LLM usage) + category breakdown; trend % = 2nd-half vs 1st-half of window; period toggle	GET /admin/analytics?period=
Admins (AdminsPage)	Table of admins: name, super/normal role, permissions, created, last login (super-admin only view)	GET /admin/admins
Audit Logs (AuditLogsPage)	Paginated (50/pg) activity feed: action, actor (admin/partner/user, color-coded), target, IP, timestamp	GET /admin/activity-logs

`adminErr(err, fallback)` normalizes backend `{detail}` error messages for every page.

4. Web push (FCM) — `src/lib/webPush.ts`

Web push works for **both users and partners** and is LIVE (per `project_web_push`). The shared `components/NotificationsToggle.tsx` appears in the user **App Settings** page (`/app-settings`) and the **partner Settings drawer**. Flow:

- `pushSupported()` checks `Notification`, `serviceWorker`, and `PushManager` exist (guards SSR / locked-down browsers). `pushStatus()` returns `'unsupported' | 'default' | 'granted' | 'denied'`.
- `enablePush()` → `Notification.requestPermission()`; if granted, `fetchAndRegisterToken()` runs.
- `fetchAndRegisterToken()` checks `Firebase.isSupported()`, gets `getMessaging(app)`, registers the **service worker** `public/firebase-messaging-sw.js` (served at site root `/firebase-messaging-sw.js`, no-cache), then `getToken(messaging, { vapidKey: VITE_FIREBASE_VAPID_KEY, serviceWorkerRegistration })`.
- The token is sent to the backend via `pushAPI.registerDevice(token)` → `POST /user/register-device?token=<fcm>` (backend resolves user OR partner). Backend `send_fcm_push(...)` (`firebase_admin`) delivers to web tokens too.
- `refreshPushIfEnabled()` silently re-fetches/re-registers the token when permission is already granted (self-heals a granted-but-unregistered state).

Config/where values live:

- `VITE_FIREBASE_VAPID_KEY` — the **public** VAPID key, set in `tefillah-web/.env` (public, safe to embed; the matching private key isn't stored anywhere because `firebase_admin` signs server-side). The value here is `<set in tefillah-web/.env>`.
- Firebase web config is **non-secret** and hardcoded in `src/lib/firebase.ts`: `project tefillah-2283c`, `authDomain tefillah-2283c.firebaseio.com`, `messagingSenderId 240620097116` (same identifiers the mobile app ships).
- CSP** for static/FCM is applied via CloudFront response-headers policy `3c3cd71b-468f-4ef7-bfbf-4efb649a29bf` (`script-src += gstatic`, `worker-src 'self'`, `connect-src +=`

FCM endpoints).

- Remaining check: a **live browser test** (the preview browser denies notifications) — grant permission on `tefillah.in`, confirm an `fcm_token` lands on the Mongo doc, send a test push.

5. Web Google sign-in — `/auth/social`

Google sign-in on web uses **Google Identity Services (GIS)** directly, not the Firebase OAuth popup (which was the piece that was broken). Gated by `GOOGLE_SIGNIN_ENABLED = true` in `src/config.ts`.

- `components/GoogleSignInButton.tsx` loads `https://accounts.google.com/gsi/client`, calls `google.accounts.id.initialize({ client_id: VITE_GOOGLE_WEB_CLIENT_ID, callback, ux_mode: 'popup' })`, and renders the official Google button. GIS returns a **Google ID token** (resp. `credential`) entirely client-side — no OAuth client secret involved.
- The callback hands the token to `lib/socialAuth.ts` → `socialSignIn(firebaseToken)` → `authAPI.socialAuth` → `POST /auth/social`. The backend verifies the token against `GOOGLE_WEB_CLIENT_ID` and returns a session (`access_token`, `user_type`, `user`). `applySession()` persists it to storage + `authStore` (mirroring `loginAsUser`) and calls `checkSwitch()`.
- **Routing after sign-in** (in the button + `socialAuth`): if the profile `isIncomplete` (missing phone/city/country) → `/complete-profile` (email/name/agent passed as query params, handled by `CompleteProfilePage` → `completeSocialProfile` → `POST /auth/social/complete`); else if not verified → `/verify`; else → `/partner/dashboard` (partners) or `/home` (users).
- There is also a legacy Firebase-popup helper `signInWithGoogle()` in `lib/firebase.ts` (returns a Firebase ID token via `signInWithPopup`), but the live web button uses GIS.
- **The one console requirement:** the OAuth **Web** client (`VITE_GOOGLE_WEB_CLIENT_ID`) must list this site under **"Authorized JavaScript origins"** in Google Cloud Console. The actual client-id value is `<set in tefillah-web/.env / build env>`; also see memory `project_google_signin_web` (Firebase authorized domains must include `tefillah.in`).

Key file paths (quick reference)

- Router: `D:/tefillah-fixed/tefillah-fixed/tefillah-web/src/App.tsx`
- Public design tokens: `.../tefillah-web/src/styles/globals.css`
- Admin design tokens: `.../tefillah-web/src/admin/admin.css`
- Admin routing / auth / API: `.../src/admin/AdminApp.tsx`, `adminAuth.tsx`, `AdminProtectedRoute.tsx`, `adminApi.ts`
- Admin shell + logo: `.../src/admin/AdminLayout.tsx`, `AdminLogo.tsx`, `adminUi.tsx`
- Admin pages: `.../src/admin/pages/{Dashboard,Users,Partners,Prayers,Analytics,Admins,AuditLogs}Page.tsx`
- Web push: `.../src/lib/webPush.ts`, `firebase.ts`, `.../public/firebase-messaging-sw.js`
- Google sign-in: `.../src/components/GoogleSignInButton.tsx`, `.../src/lib/socialAuth.ts`
- Public API client: `.../src/api/client.ts`

Deploy (from `project_web_deploy`)

`tefillah.in` is **AWS S3 + CloudFront** (not Vercel/Cloudflare — `tefillah-web/DEPLOY.md` is stale). From `tefillah-web/`:

1. `npm run build` → `dist/`
2. `aws s3 sync dist/ s3://tefillah-web-prod/ --delete --exclude index.html --exclude firebase-messaging-sw.js`
3. `aws s3 cp dist/index.html s3://tefillah-web-prod/index.html --content-type "text/html" --cache-control "no-cache, no-store, must-revalidate"`
4. `aws cloudfront create-invalidation --distribution-id E20DJ1IDF5M5MD --paths "/*"`

Bucket `tefillah-web-prod`; distribution `E20DJ1IDF5M5MD` (`dc71sb5z88jh2.cloudfront.net`) serves `tefillah.in` + `www.tefillah.in`. Verify the live bundle hash with `curl -s https://tefillah.in/ | grep -oE "index-[A-Za-z0-9_-]+\\.js"`. Watch for local **AWS clock skew** at deploy time (`reference_aws_clock_skew`).

CHAPTER 6

Credentials, Environment Map & Deployment Runbooks

Overview: three components, three config surfaces

Tefillah is three deployable pieces, each with its own configuration surface. Know which surface a variable lives in before you touch it — editing the wrong one is the #1 cause of "I changed it and nothing happened."

Component	What it is	Config surface (source of truth)
Backend (FastAPI)	backend/server.py on Elastic Beanstalk	EB environment vars in prod; backend/.env is dev-only
Mobile app (Expo/RN)	frontend/, built via EAS	EXPO_PUBLIC_* (baked into the build); some values hardcoded in app.json
Web site + /admin (Vite/React)	tefillah-web/, on S3+CloudFront	VITE_* at build time + a hardcoded Firebase block

The single most important rule: the backend's real production secrets (MONGO_URL, JWT_SECRET, GEMINI_API_KEY, etc.) do **NOT** live in any file on disk. They live in the **Elastic Beanstalk environment** tefillah-api-prod-v2. The local backend/.env points at localhost:27017 with a throwaway JWT secret — tokens minted with it will NOT validate against prod, and it will NOT connect to the prod database. Never copy local .env values expecting them to work in production.

To read the real prod values (they are secrets — keep them only in the session scratchpad, never commit):

```
aws elasticbeanstalk describe-configuration-settings \
  --region ap-south-1 \
  --application-name tefillah-api \
  --environment-name tefillah-api-prod-v2 \
  --query "ConfigurationSettings[0].OptionSettings[?Namespace=='aws:elasticbeanstalk:application:environment']"
```

1. Credentials & environment-variable map

1a. Backend variables (backend/.env names; prod values in EB env tefillah-api-prod-v2)

Every value below is masked here. Where it says "EB environment," that is the live production value; the backend/.env copy is the dev-only version.

Var	Purpose	Where the value lives	How to rotate
MONGO_URL	MongoDB Atlas connection URI (cluster tefillah.l2f7ieg.mongodb.net, user tefillah-app)	EB env (prod) / backend/.env = localhost (dev)	Atlas console → Database Access → edit tefillah-app password → update EB env var, redeploy
DB_NAME	Database name. Prod = tefilah (one L)	EB env / backend/.env	Rarely; changing it repoints to a different DB

Var	Purpose	Where the value lives	How to rotate
JWT_SECRET	HMAC key signing all auth tokens	EB env (prod) / backend / .env (dev throwaway)	Generate new random string, set in EB env, redeploy — invalidates all existing sessions
JWT_ALGORITHM	HS256	EB env / .env	Don't change
JWT_EXPIRATION_DAYS	Token lifetime (7)	EB env / .env	Edit value, redeploy
ADMIN_SECRET	One-time secret to bootstrap the first admin	EB env / .env	Rotate after first admin created
LLM_PROVIDER	Which LLM path: gemini (primary) / openrouter / ollama	EB env / .env	Edit value
GEMINI_API_KEY	Google AI Studio key (prayer generation)	EB env / .env; issued at aistudio.google.com	AI Studio → API keys → revoke+create → update EB env
GEMINI_MODEL	gemini-2.5-flash	EB env / .env	Edit value
OLLAMA_BASE_URL / OLLAMA_MODEL	Local LLM fallback (dev only)	.env	n/a in prod
OPENROUTER_API_KEY / OPENROUTER_MODEL	Legacy LLM fallback	EB env / .env; openrouter.ai	OpenRouter dashboard → rotate key
RESEND_API_KEY	Resend email API key (all system email)	EB env / .env; resend.com dashboard	Resend → API Keys → revoke+create → update EB env
SENDER_EMAIL	From-address; admin@tefillah.in	EB env (also re-set by deploy script) / .env	Edit value; must be a verified Resend sender
FIREBASE_PROJECT_ID	tefillah-2283c	EB env / .env	Stable
FIREBASE_WEB_API_KEY	Verifies Firebase ID tokens on social login	EB env / .env; Firebase console	Firebase → Project settings → rotate
FIREBASE_ADMIN_CREDENTIALS	Service-account JSON for FCM push (sending)	EB env / bundled firebase-credentials.json	Firebase → Service accounts → generate new private key → replace
GOOGLE_WEB_CLIENT_ID	OAuth web client id for Google sign-in	EB env / .env; Google Cloud console	GCP → Credentials → rotate client secret (id stays)
RATE_LIMIT_REQUESTS / RATE_LIMIT_WINDOW	Per-IP throttle (100 / 60s)	EB env / .env	Edit value, redeploy
ALLOWED_ORIGINS	CORS allowlist (dev has localhost:3000/8081/19006)	EB env / .env	Add prod origins in EB env

How to edit an EB env var (this is a "rotate/change" for any backend var):

```
aws elasticbeanstalk update-environment \
  --region ap-south-1 \
  --environment-name tefillah-api-prod-v2 \
  --option-settings "Namespace=aws:elasticbeanstalk:application:environment,OptionName=GEMINI_API_KEY,Value=NEW_VALUE_HERE"
```

The environment restarts and picks up the new value. No code redeploy needed for an env-only change.

1b. Mobile variables (EXPO_PUBLIC_*, baked into the EAS build)

EXPO_PUBLIC_* vars are **compiled into the app binary at build time** — they are NOT secret at runtime (anyone can extract them) and they only change when you rebuild. Firebase web/OAuth client config is inherently public;

the security comes from the backend verifying tokens, not from hiding these.

Var	Purpose	Where set	How to change
EXPO_PUBLIC_BACKEND_URL	API base; prod = <code>https://api.tefillah.in</code>	EAS build env / <code>frontend/.env</code> (dev = <code>localhost:8001</code>)	Set in EAS env, rebuild
EXPO_PUBLIC_FIREBASE_API_KEY	Firebase web API key	EAS build env; Firebase console	Rotate in Firebase, rebuild
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN	<code>tefillah-2283c.firebaseio.com</code>	EAS build env	Stable
EXPO_PUBLIC_FIREBASE_PROJECT_ID	<code>tefillah-2283c</code>	EAS build env	Stable
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET	Firebase storage bucket	EAS build env	Stable
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	FCM sender id (240620097116)	EAS build env	Stable
EXPO_PUBLIC_FIREBASE_APP_ID	Firebase app id	EAS build env	Stable
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID	OAuth web client id for native Google sign-in	EAS build env; GCP Credentials	Rotate in GCP, rebuild

Fixed identifiers baked into `frontend/app.json` (not env): app name `Tefillah`, slug `tefillah`, version `1.0.0`, Android package `com.tefillah.app`, versionCode `4` (**ignored** — `appVersionSource: remote + autoIncrement`), Expo owner `tefillah`, EAS projectId `22c2d81a-4192-419e-973e-e2112b437e07`.

1c. Web variables (VITE_* + hardcoded Firebase block)

Only three values come from `VITE_*` at build time; the rest of the Firebase web config is **hardcoded** in `tefillah-web/src/lib/firebase.ts` (public client config — safe to ship).

Var	Purpose	Where set	How to change
VITE_API_URL	API base; prod = <code>https://api.tefillah.in/api</code>	<code>tefillah-web/.env</code> at build	Edit <code>.env</code> , rebuild+redeploy
VITE_GOOGLE_WEB_CLIENT_ID	OAuth web client id (Google sign-in on site)	<code>.env</code> at build; GCP	Rotate in GCP, rebuild
VITE_FIREBASE_VAPID_KEY	Web-push VAPID public key (FCM web push cert)	<code>.env</code> at build; Firebase → Cloud Messaging → Web Push certificates	Regenerate cert in Firebase, rebuild

Hardcoded in `firebase.ts` (public, not secrets): `apiKey`, `authDomain: tefillah-2283c.firebaseio.com`, `projectId: tefillah-2283c`, `messagingSenderId: 240620097116`, `appId: 1:240620097116:web:...` If you migrate Firebase projects, edit this file directly.

2. External accounts (who to log into for what)

Account	Login identity	What it controls
AWS (account 020262236044)	your AWS console user	Elastic Beanstalk (tefillah-api / tefillah-api-prod-v2), S3 tefillah-web-prod, CloudFront E20DJ1IDF5M5MD, Route 53 zone Z06246302MVTLD5P84FBW, IAM. Region: ap-south-1 (Mumbai)
MongoDB Atlas	Atlas account	Cluster tefillah (tefillah.l2f7ieg.mongodb.net), DB user tefillah-app, network allowlist, backups
Google Cloud / Firebase	Google acct → project tefillah-2283c	OAuth clients (web client id), FCM push, Firebase Auth (Google sign-in), web-push VAPID cert
Google Workspace	Sign in as admin@tefillah.in (this is the tefillahprayerapp@gmail.com account with the custom domain attached; password unchanged)	admin.google.com super-admin; the admin@tefillah.in mailbox (system-email From-address); also the identity for EAS/Play
Resend	Resend account	Transactional email sending; domain tefillah.in + send.tefillah.in verified
Expo / EAS	tefillah account = tefillahprayerapp@gmail.com	Android/iOS builds, build credentials (keystore "Build Credentials 8F-8Eh2clT"), OTA. Project owner tefillah

Same underlying Google account (tefillahprayerapp@gmail.com) **is Workspace admin, Firebase owner, and the EAS tefillah account.** Its sign-in is now admin@tefillah.in.

3. Deployment runbooks

3a. Backend → Elastic Beanstalk

The deploy script is **self-verifying**. It repackages tefillah-api-v4.zip from the CURRENT backend/server.py on every run and refuses to ship if the packaged code doesn't byte-match the source or still contains Twilio references. Run from the repo root:

```
cd /d/tefillah-fixed/tefillah-fixed
bash deploy-backend.sh
```

What it does, in order (from deploy-backend.sh):

1. Copies backend/server.py into the _eb_build/ staging dir (Procfile, requirements.txt, firebase-credentials.json, .ebextensions/ already staged there).
2. Computes md5 of source; aborts if any Twilio reference is present.
3. Zips the six bundle members with Python (no zip binary needed).
4. **Hard gate:** md5 of the zipped server.py must equal the source md5, else abort.
5. Uploads to elasticbeanstalk-ap-south-1-<account> S3 bucket.
6. create-application-version (label v33-security-hardening-<timestamp>).
7. update-environment on tefillah-api-prod-v2, also re-asserting SENDER_EMAIL=admin@tefillah.in.
8. Polls health until Ready, then smoke-tests the health endpoint.

Verify after deploy (script does this, but confirm manually):

```
curl -s https://api.tefillah.in/api/health
```

Expect HTTP 200. If health goes red, check EB env events:

```
aws elasticbeanstalk describe-events --region ap-south-1 \
  --environment-name tefillah-api-prod-v2 --max-records 20
```

3b. Web (tefillah.in + /admin) → S3 + CloudFront

The live site is S3 + CloudFront, NOT Vercel/Cloudflare — `tefillah-web/DEPLOY.md` is stale, ignore it. Run from `tefillah-web/`:

```
cd /d/tefillah-fixed/tefillah-fixed/tefillah-web
npm run build
# sync everything EXCEPT the two files that must never be cached long
aws s3 sync dist/ s3://tefillah-web-prod/ --delete \
  --exclude index.html --exclude firebase-messaging-sw.js
# push index.html with no-cache so users get the new bundle immediately
aws s3 cp dist/index.html s3://tefillah-web-prod/index.html \
  --content-type "text/html" \
  --cache-control "no-cache, no-store, must-revalidate"
# invalidate the CDN
aws cloudfront create-invalidation \
  --distribution-id E20DJ1IDF5M5MD --paths "/"
```

`E20DJ1IDF5M5MD` (domain `dc71sb5z88jh2.cloudfront.net`) serves `tefillah.in` + `www.tefillah.in` from bucket `tefillah-web-prod`. Note the same bucket also serves `/avatars/*` (profile photos), which is why you `--exclude` nothing under `avatars` but let `--delete` prune only stale build assets.

Verify the freshly-built bundle hash is live:

```
curl -s https://tefillah.in/ | grep -oE "index-[A-Za-z0-9_-]+\..js"
```

Compare that hash against the file in your local `dist/assets/`. If they match, the deploy is live (invalidation propagation is usually under a minute).

3c. Mobile → EAS build

Builds MUST run as the tefillah Expo account (`tefillahprayerapp@gmail.com`). The CLI session sometimes drifts to a personal account and fails with `Entity not authorized: AppEntity[22c2d81a-...]`. Fix login first:

```
npx eas-cli logout
npx eas-cli login          # sign in as tefillahprayerapp@gmail.com
npx eas-cli whoami         # must print: tefillah
```

(The user's PowerShell has no global `eas` on PATH — use `npx eas-cli`. Login is interactive; it can't be automated for you.)

Then build from `frontend/`:

```
cd /d/tefillah-fixed/tefillah-fixed/frontend

# Installable APK for testing / internal distribution:
eas build --platform android --profile preview --non-interactive --no-wait

# Play Store AAB (production, auto-increments versionCode remotely):
eas build --platform android --profile production --non-interactive --no-wait
```

From `frontend/eas.json`: `preview` → `buildType: apk`, internal distribution; `production` → `buildType: app-bundle`, `autoIncrement: true`, `appVersionSource: remote`, so the `versionCode` in `app.json` is ignored

— EAS owns the version counter. Preview builds are signed with keystore "Build Credentials 8F-8Eh2cIT".

3d. DNS → Route 53

DNS lives in Route 53 zone `Z06246302MVTLD5P84FBW`. To edit records:

```
aws route53 change-resource-record-sets --hosted-zone-id Z06246302MVTLD5P84FBW \
  --change-batch file://change.json
```

Key records: `tefillah.in/www` → CloudFront `E20DJ1IDF5M5MD`; `api.tefillah.in` → EB env; mail records per section 4 below.

4. Google Workspace email specifics (and the Resend coexistence rule)

`admin@tefillah.in` is a **Google Workspace** mailbox (the `tefillahprayerapp@gmail.com` account with the domain attached). System email is sent **as** Tefillah `<admin@tefillah.in>` via Resend (backend `SENDER_EMAIL=admin@tefillah.in`).

Mail DNS records in Route 53 (`Z06246302MVTLD5P84FBW`):

Record	Value	Purpose
Root MX	<code>1 smtp.google.com</code>	Inbound mail → Google Workspace
Root SPF TXT	<code>v=spf1 include:_spf.google.com ~all</code>	Authorizes Google to send
<code>google._domainkey</code> TXT	Google DKIM (2048-bit, split into 2 quoted strings for the 255-char limit)	Google DKIM signing
<code>_dmarc</code> TXT	<code>p=quarantine, relaxed</code>	DMARC policy

CRITICAL — do NOT touch the Resend records. App verification / reset / notification emails depend on them, and they use the `send.` subdomain so the root MX/SPF pointing at Google does not affect them:

- `send.tefillah.in` MX → `feedback-smtp...amazonses.com`
- `send.tefillah.in` SPF → `include:amazonses.com`
- `resend._domainkey` DKIM

Google mail and Resend mail each pass DMARC via their own aligned DKIM. Harmless leftovers you may clean up later: `cf2024-1._domainkey` (old Cloudflare DKIM), `email.secureserver.net` / `_autodiscover` / `_domainconnect` (old GoDaddy). `grievance@tefillah.in` is a separate legal contact — leave it.

Resend gotcha (load-bearing): `api.resend.com` is behind Cloudflare, which silently blocks non-browser User-Agents with HTTP 403 / "error code 1010." The backend sends a Chrome `RESEND_USER_AGENT` header on every Resend call — **do not remove it**. If email silently breaks, test from a shell with vs without a Mozilla/5.0 UA; a 1010 means Cloudflare is blocking again.

5. The AWS clock-skew gotcha (read this before every deploy)

This Windows machine's clock periodically drifts **behind** real time (seen ~5h behind). AWS SigV4 allows only 15 minutes of skew, so every AWS CLI call (deploy, S3 sync, CloudFront invalidation) then fails with:

```
SignatureDoesNotMatch ... Signature expired: <local time> is now earlier than <aws time>
```

Diagnose: compare `date -u` against the AWS time in the error message.

Fix (only the user can do this — it needs admin): Windows taskbar clock → *Adjust date and time* → **Sync now** (or toggle "Set time automatically" off/on). `w32tm /resync` and `Set-Date` both return "Access is denied" without admin, and there is no boto core flag to override the signing clock.

Confirm fixed, then retry the deploy:

```
aws sts get-caller-identity
```

If a deploy that worked yesterday suddenly fails with signature-expired, suspect clock drift **first** — it is almost always this, not a credentials problem.

CHAPTER 7

The "Become a Master" Playbook

The "Become a Master" Playbook

This chapter is the one you keep open while you work. It answers three questions fast: **Where do I edit X?**, **Something broke — how do I fix it myself?**, and **How do I run the day-to-day operations?** Everything here is verified against the live tree at `D:/tefillah-fixed/tefillah-fixed`.

Three trees make up the whole product, and almost every change lands in exactly one of them:

- `backend/` — one FastAPI file, `backend/server.py` (~5,122 lines). Every route, the LLM call, all email, all auth, all admin logic lives here. Deployed to AWS Elastic Beanstalk via `deploy-backend.sh`.
- `tefillah-web/` — the Vite/React public site (`src/pages/*`) **and** the `/admin` panel (`src/admin/*`). Deployed to S3 + CloudFront.
- `frontend/` — the Expo/React Native mobile app (`app/**`). Built into an Android `.aab`/APK with EAS.

1. WHERE TO EDIT WHAT — the master lookup table

Task	File(s)	How
Change LLM provider	<code>backend/server.py</code> L71 <code>LLM_PROVIDER</code>	Set env <code>openrouter \</code> <code>LLM_PROVIDER =</code> <code>gemini \</code> <code>ollama</code> (default <code>gemini</code>). Value lives in EB env.
Change Gemini model	<code>backend/server.py</code> L84 <code>GEMINI_MODEL</code>	Env <code>GEMINI_MODEL</code> (default <code>gemini-2.5-flash</code>). Set in <code>backend/.env</code> / EB env.
Change OpenRouter model	<code>backend/server.py</code> L79 <code>OPENROUTER_MODEL</code>	Env <code>OPENROUTER_MODEL</code> (default <code>deepseek/deepseek-chat-v3-0324</code>).
Change Ollama model/host	<code>backend/server.py</code> L75 <code>OLLAMA_MODEL</code> , <code>OLLAMA_BASE_URL</code>	Env vars; local-only provider.
Edit the AI prayer prompt	<code>backend/server.py</code> around the <code>generate_* LLM</code> builders (L900–1110)	Edit the system/prompt text; provider dispatch is at L1107 (if <code>LLM_PROVIDER == "gemini"</code>).
Edit an account email's HTML shell	<code>backend/server.py</code> L861 <code>_account_email_html(heading, paragraphs)</code>	Change wrapper markup/colors; used by every notice.
Send/change an account notice email	<code>backend/server.py</code> L881 <code>send_account_notice(to_email, heading, subject, paragraphs)</code>	Call it or edit copy; it renders via <code>_account_email_html</code> .

Task	File(s)	How
Change the sender email	env SENDER_EMAIL (default forced to admin@tefillah.in by deploy-backend.sh L92)	Set in EB env; the deploy script re-sets it on every deploy.
Rotate the Resend API key	env RESEND_API_KEY	Value in backend/.env / EB env. Note the browser-UA requirement (see troubleshooting).
Add / adjust an API endpoint	backend/server.py, decorators@api_router.<verb>("/...")	All routes hang off api_router (prefix /api). Add function + decorator; redeploy.
Add an admin permission gate	backend/server.py L789 check_admin_permission(admin, "<perm>")	Call at the top of the handler. Valid perms: manage_users, manage_partners, manage_prayers, view_analytics, export_data; all/super_admin bypasses.
Harden a login flow (lockout)	backend/server.py L236 check_login_lockout, L247 record_failed_login, L266 progressive_login_delay	Use key pattern user:<email> / partner:<email> / admin:<email>.
Change admin theme colors	tefillah-web/src/admin/admin.css (token block near top)	Edit the CSS custom properties (near-black canvas + gold hero + cyan/violet/emerald).
Change the breathing admin logo	tefillah-web/src/admin/AdminLogo.tsx + breathe keyframes in admin.css	Swap the Ionicons flame or tweak the breathe animation timing.
Change the public-site logo / breathe	tefillah-web/src/** Logo component + src/styles/globals.css breathe keyframes	globals.css was reconstructed post-crash — tokens are best-effort, fine-tune here.
Edit public site copy (home, about, legal)	tefillah-web/src/pages/*.tsx (e.g. HomePage.tsx, AboutPage.tsx, PrivacyPage.tsx, TermsPage.tsx)	Plain JSX text edits; rebuild + redeploy web.
Edit the public delete-account page	tefillah-web/src/pages/DeleteAccountPage.tsx	This is the Play-Console "deletion URL" target (tefillah.in/delete-account).

Task	File(s)	How
Edit an admin page (users/partners/etc.)	tefillah-web/src/admin/pages/*.tsx (UsersPage, PartnersPage, PrayersPage, AnalyticsPage, AuditLogsPage, AdminsPage, DashboardPage)	React pages; API calls via src/admin/adminApi.ts.
Add/adjust an admin API call	tefillah-web/src/admin/adminApi.ts	Central admin fetch client; add method mirroring the backend route.
Edit a mobile screen	frontend/app/(auth)/*, frontend/app/(main)/*, frontend/app/(partner)/*	Expo Router file-based screens (e.g. (main)/prayer.tsx, (main)/history.tsx, (partner)/dashboard.tsx).
Change mobile API base / client	frontend/src/api/client.ts	Central mobile API client (reconstructed post-crash — holds switch/report/blockUser/flagAiContent/unregisterToken).
Change app icon / splash / colors	frontend/app.json (icon, expo-splash-screen plugin, #0f0f1a bg, #d4af37 accent)	Replace files in frontend/assets/images /; edit plugin config.
Change Android permissions	frontend/app.json → android.permissions / android.blockedPermissions	Add/remove; store audit blocks SYSTEM_ALERT_WINDOW, READ_EXTERNAL_STORAGE, RECEIVE_BOOT_COMPLETED, SCHEDULE_EXACT_ALARM.
Bump the Android version	frontend/app.json android.versionCode (currently 4) and EAS remote	eas.json uses appVersionSource: remote + autoIncrement on production — EAS owns the build number; keep app.json in sync for local clarity.
Change bundle/package id	frontend/app.json android.package / ios.bundleIdentifier = com.tefillah.app	Changing this is a new app in the stores — avoid.
Change CORS / allowed origins	env ALLOWED_ORIGINS (read in server.py)	Comma-separated; set in EB env.
Toggle API docs (Swagger)	env ENABLE_DOCS	Off in prod by default.

Task	File(s)	How
Rotate JWT secret	env JWT_SECRET (+ guard ALLOW_DEFAULT_JWT_SECRET)	Value in EB env; see backend-won't-boot fix below.
Rotate the admin bootstrap secret	env ADMIN_SECRET (server.py L121)	Used only by POST /admin/first-admin; must be strong in prod.
Firebase / Google client ids	env FIREBASE_PROJECT_ID, FIREBASE_WEB_API_KEY, GOOGLE_WEB/ANDROID/IOS_CLIENT_ID; frontend/google-services.json	Non-secret ids; service-account JSON is firebase-credentials.json (secret).
Deploy the backend	deploy-backend.sh (repo root)	bash deploy-backend.sh — rebuilds the zip from current server.py, md5+Twilio gate, EB app tefillah-api, env tefillah-api-prod-v2, region ap-south-1.
Deploy the web	tefillah-web/ build + S3 sync	See Operations runbook step; CloudFront E20DJ1IDF5M5MD, bucket tefillah-web-prod.
Build the mobile app	frontend/ + EAS	eas build -p android --profile production (.aab) under the tefillah Expo account.

2. SELF-FIX TROUBLESHOOTING PLAYBOOK

Read this as **symptom** → **likely cause** → **fix**. Most Tefillah incidents are one of these.

2a. The RECURRING crash-corruption (do this FIRST after any Claude/process crash)

A process crash on/around a "task stopped / previous process exited" boundary has **twice** corrupted (null-byte) or deleted source files and node_modules in both frontend/ and tefillah-web/. Live services are unaffected (they predate the crash). After ANY crash, before building anything:

```
# 1) Find null-byte-corrupted files (they contain \0)
cd D:/tefillah-fixed/tefillah-fixed
grep -rLP "\x00" backend frontend/app frontend/src tefillah-web/src 2>/dev/null

# 2) Find deleted tracked files (frontend/ and tefillah-web/ are git repos)
cd frontend      && git status --short | grep '^ D'
cd ../tefillah-web && git status --short | grep '^ D'
```

Recovery, in order of preference:

1. git checkout HEAD -- <path> for tracked files (fast, exact). Note HEAD may be **stale** — verify the restored file has the newest logic (e.g. client.ts must contain

`report/blockUser/flagAiContent/unregisterToken`).

2. `backend/server.py` specifically: restore from `_eb_build/server.py` (the deployed copy) if the working copy is corrupted.
3. Untracked files with no backup (e.g. `profile-settings.tsx`, reconstructed web pages) must be **rebuilt** — cross-check against `project_crash_corruption_2026-07` memory for exactly what each held.
4. Corrupted `node_modules`: `rm -rf node_modules && npm install` in the affected tree.

Verify recovery: mobile `npx tsc --noEmit clean` **and** `npx expo export --platform android` bundles all modules (was 1727) with no errors; web `npm run build clean`.

2b. Backend won't boot — JWT fail-closed guard

- **Symptom:** backend raises `RuntimeError` on startup mentioning the default JWT secret.
- **Cause:** the v33 fail-closed guard (`server.py` ~L52–61) refuses to boot on a remote Mongo (`mongodb+srv` / non-localhost) if `JWT_SECRET` is still the default.
- **Fix (prod):** set a strong `JWT_SECRET` in the EB env. Do **not** work around it in prod.
- **Fix (local against remote Mongo, intentionally):** set `ALLOW_DEFAULT_JWT_SECRET=true`. Only for dev. Same pattern applies to `ADMIN_SECRET` (raises in production if left default).

2c. Resend email "error 1010"

- **Cause:** Cloudflare blocks the Resend call when there's no browser `User-Agent`.
- **Fix:** the Resend HTTP call must send a browser UA header. If email suddenly stops, confirm the UA header is still present on the Resend request in `server.py`. (See `project_resend_email_ua`.)

2d. AWS "signature expired" / verification token "invalid"

- **Cause:** the local Windows clock drifts **behind** real time → AWS rejects signatures as expired, and email OTP/verification tokens read as invalid due to clock skew.
- **Fix:** Windows Settings → Time & Language → Date & time → **Sync now**. If it won't sync, it needs admin rights. (See `reference_aws_clock_skew`.)

2e. Expo / mobile build errors

- **First:** `npx expo install --check` then `npx expo install --fix` (aligns package versions to the SDK).
- **Then:** `npx expo-doctor` for deeper diagnosis.
- **Metro/cache weirdness:** `npx expo start -c` (clear cache).
- **Bundle sanity before a build:** `npx expo export --platform android` — must complete with no errors.
- **EAS auth:** builds MUST run under the `tefillah` Expo account — `npx eas-cli login` as `tefillah` first (see `project_eas_build_account`).

2f. Web build errors

- Run `npm run build` in `tefillah-web/`. Post-crash, the usual culprit is a reconstructed file (`globals.css`, `webPush.ts`, `CompleteProfilePage.tsx`, `PartnerSignupPage.tsx`) with a missing import or dep. `npm install` (firebase in particular was re-added post-crash), then rebuild.

2g. `git push` "repository not found"

- **Cause:** the cached GitHub credential is the wrong account.
- **Fix:** confirm the remote (`git remote -v`) and the credential in use. On Windows, clear the wrong entry in **Credential Manager** (or `gh auth switch`) so the correct GitHub account authenticates.

2h. Deploy shipped stale code

- **Cause:** the old `deploy-backend.sh` uploaded a **prebuilt** zip with no repackage — it could silently ship pre-v33 code (Twilio back, hardening gone).

- **Fix:** already fixed. The current script rebuilds the zip from live `server.py` and **hard-gates** on `zip server.py md5 == backend/server.py md5` plus a Twilio-free grep; it aborts otherwise. If a deploy aborts with "FATAL: ... md5 ... != source", that gate just saved you — re-stage and re-run rather than bypassing it.

3. OPERATIONS RUNBOOK

3a. Create the FIRST admin (one-time bootstrap)

POST `/api/admin/first-admin` with header `X-Admin-Secret: <ADMIN_SECRET>` and body `{name, email, password}`.

- It **hard-fails with 409 once any admin exists** (`server.py` L2894) — genuinely one-time.
- The created admin is `is_super_admin: true, permissions: ["all"]`.

```
curl -X POST https://api.tefillah.in/api/admin/first-admin \
-H "X-Admin-Secret: <ADMIN_SECRET from EB env>" \
-H "Content-Type: application/json" \
-d '{"name": "Suraj", "email": "you@example.com", "password": "<strong>"}'
```

3b. Create OTHER admins (after bootstrap)

Only a **super_admin** can. Two endpoints exist:

- POST `/api/admin/admins` (`create_admin`, L2977) — default perms `["manage_prayers", "manage_partners", "manage_users", "view_analytics"]`.
- POST `/api/admin/admins` with permissions payload (`create_admin_with_permissions`, L3976) — pick a subset.
- Change an existing admin's perms: PUT `/api/admin/admins/{admin_id}/permissions` (L4011).
- In the UI: `/admin` → **Admins** page (`AdminsPage.tsx`).

3c. Approve / manage a partner

- UI: `/admin` → **Partners** (`PartnersPage.tsx`) → activate/suspend/edit. Backend routes are the `manage_partners`-gated handlers (L3380+). Partner status is re-checked on **every** request, so suspending takes effect immediately.

3d. Handle a reported prayer / block a user

Partners do this from their dashboard; you can too:

- Report + release a request: POST `/api/partner/requests/{prayer_id}/report` (L2604).
- Block a user: POST `/api/partner/block-user` (L2645) — capped at `MAX_BLOCKED_USERS=5000`.
- Flag AI-generated content: POST `/api/prayer/{prayer_id}/flag` (L2691).
- Admin side: `/admin` → **Prayers** (`manage_prayers`) to view/assign/unassign.

3e. Send an announcement / push / email blast

- In-app notification (targeted or broadcast): POST `/api/admin/notifications` (`create_notification`, L3868). Users read via `/api/user/notifications`.
- Push notification: POST `/api/admin/push-notification` (`send_push_notification`, L4218) → FCM. Mobile push uses the **native FCM device token** (`getDevicePushTokenAsync`), not the Expo token.
- Email broadcast: POST `/api/admin/email-broadcast` (`send_email_broadcast`, L4662) — runs in background, logs `email_broadcast_started/completed`.

3f. Export data / read audit logs

- CSV export: GET `/api/admin/export-csv/{data_type}` (L3925, gated `export_data`).

- LLM usage logs: `GET /api/admin/llm-logs` (L3799, `view_analytics`).
- Activity/audit logs: `GET /api/admin/activity-logs` (L3832, `view_analytics`) or UI **Audit Logs** page.

4. PLAY STORE / APP STORE SUBMISSION CHECKLIST

Backend account-takeover holes are closed, deletion + UGC moderation shipped, permissions cleaned, AI content is labeled. Ultra pre-Play review verdict was **GO for the Play Store .aab**, **zero mobile blockers**. What remains is Console paperwork and the two build commands (both on your explicit command only).

Google Play (Android) — ready:

1. Build the bundle: `eas build -p android --profile production` (under the `tefillah` Expo account → `.aab`).
2. **Data Safety form** — declare: location (partner matching), photos (profile), push token, AI-generated content, account deletion. Do **NOT** declare SMS (Twilio removed permanently).
3. **Privacy policy URL** → `tefillah.in/privacy` (in Play Console → App content).
4. **Account deletion URL** → confirm `tefillah.in/delete-account` resolves (public `DeleteAccountPage.tsx`).
5. **Content rating** questionnaire.
6. **Screenshots** (phone; feature graphic).
7. Upload the `.aab` manually to the release track.

Apple App Store (iOS) — one remaining item:

- **Sign in with Apple** is required *because* the app offers Google sign-in on iOS. This is the only known iOS blocker. Same Data-Safety-equivalent (App Privacy) and the same privacy/deletion URLs apply. `ITSAppUsesNonExemptEncryption=false` is already set in `app.json`.

Deploy gates (held for review): `deploy-backend.sh` (backend v33) and the production `.aab` are **staged and verified but not shipped** — you run them when ready.

5. MASTER-IT LEARNING PATH

Read the code in this order (each builds on the last):

1. `C:/Users/suraj/.claude/projects/D--tefillah-fixed/memory/project_overview.md` — the map.
2. `backend/server.py` **top ~130 lines** — every env var, the JWT/ADMIN_SECRET guards, LLM config. This is the config surface of the whole system.
3. `backend/server.py` route bodies — search `@api_router` and skim each handler group: `auth` → `prayer` → `partner` → `admin`. This one file is the backend.
4. `tefillah-web/src/pages/*` — the public user journey (Landing → Signup → Verify → Home → Prayer → Confirmation → History).
5. `tefillah-web/src/admin/*` — `adminApi.ts` (what the panel can do) then the pages.
6. `frontend/app/**` + `frontend/src/api/client.ts` — the mobile mirror of the web flows.
7. `deploy-backend.sh`, `project_web_deploy.md`, `project_prod_config.md` — how it all reaches production.

Daily / periodic maintenance habits:

- **After every crash:** run the 2a scan (`grep -rLP "\x00" + git status --short` in both JS trees) **before** touching anything. This is the single most important habit for this repo.
- **Before any build:** web `npm run build`; mobile `npx expo install --check` then `npx expo export --platform android`. Green both gates.
- **Before any backend deploy:** let `deploy-backend.sh`'s md5 + Twilio gates run — never bypass them.

- **Secrets stay in EB env / backend/.env / provider consoles** — never commit real values; the repo should only ever hold masked references.
- **Clock check** when AWS or OTP acts up: Windows "Sync now".
- **Keep app.json versionCode and the EAS remote build number coherent** so store uploads never collide.

CHAPTER 8

Appendix — Command Cheat-Sheet

Backend

```
# Deploy backend v33 to Elastic Beanstalk (rebuilds + hash-verifies the zip)
cd D:/tefillah-fixed/tefillah-fixed
bash deploy-backend.sh

# Health check
curl https://api.tefillah.in/api/health

# Syntax-check before deploying
python -m py_compile backend/server.py
```

Web (public site + admin panel)

```
cd D:/tefillah-fixed/tefillah-fixed/tefillah-web
npm run build
aws s3 sync dist/ s3://tefillah-web-prod/ --delete --exclude index.html --exclude
  firebase-messaging-sw.js
aws s3 cp dist/index.html s3://tefillah-web-prod/index.html --cache-control "no-cache"
aws cloudfront create-invalidation --distribution-id E20DJ1IDF5M5MD --paths "/*"

# Local dev (admin at /admin/login)
npm run dev
```

Mobile (Expo / EAS)

```
cd D:/tefillah-fixed/tefillah-fixed/frontend
npx eas-cli login                # sign in as the 'tefillah' account
npx expo-doctor                  # project health check (aim for 18/18)
npx expo install --fix           # align dependencies to the SDK
npx expo export --platform android # local Hermes bundle check

eas build -p android --profile preview    # installable APK
eas build -p android --profile production # Play Store AAB
```

Git

```
cd D:/tefillah-fixed/tefillah-fixed
git status
git add -A && git commit -m "your message"
git push
```

Fast fixes

```
# "expo errors" / dependency drift
npx expo install --check && npx expo-doctor

# node_modules corrupted (undici SyntaxError, babel errors, etc.)
rm -rf node_modules && npm install

# AWS "signature expired" -> Windows clock is behind:
#   Settings > Time & Language > Date & time > Sync now

# git push "repository not found" -> wrong cached GitHub account:
#   remove the git:https://github.com entry in Windows Credential Manager, push again, sign in
#   as the right account
```